KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

ASSIGNMENT II (CASE STUDY)

EXPLORATORY DATA ANALYSIS AND VISUALIZATION

Case Study: Exploratory Data Analysis and Visualization of an E-Commerce Purchasing Intention Dataset

Detail	Information
Course Code and Title	U21ADP05 - Exploratory Data Analysis and Visualization
Student Name	Jayavanth Palanisamy
Roll Number	<<ROLL NO>>
Department	Artificial Intelligence and Data Science
Degree & Year / Semester	B.Tech. AI & DS / III-YR / V Semester
Academic Year	2026-2027 (Odd Semester)
Submission Date	15 October 2026
Public GitHub Repository	https://github.com/jayavanthpalanisamy-cmd/EDA

Abstract

This study presents an end-to-end exploratory data analysis, probabilistic distribution modeling, and deep learning classification on the Online Shoppers Purchasing Intention dataset from the UCI repository, comprising 12,330 sessions across 18 behavioral and temporal attributes. Following Ben Fry's seven-stage visualization pipeline, we examine digital browsing behavior, session dwell durations, and Google Analytics indicators to model customer purchase intention. Discrete and continuous distribution fitting reveals severe overdispersion in pageviews (variance-to-mean ratio of 62.33) and marked right-tail skewness in durations, where Negative Binomial and Log-Normal models significantly outperform Poisson and Gaussian baselines. Addressing the 5.46:1 class imbalance via a leakage-free 70/15/15 stratified partition, we benchmark Cost-Sensitive Logistic Regression, Random Forest, and a custom PyTorch Tabular Deep MLP. Validation threshold tuning optimizes F1-scores, establishing Random Forest (PR-AUC 0.7506) and Tabular MLP (PR-AUC 0.7320) as robust predictors driven primarily by page values. (Word count: 138 words)

Keywords: Keywords: Exploratory Data Analysis, Probability Distribution Modeling, Overdispersion, Ben Fry Taxonomy, Class Imbalance, Deep Tabular MLP, Threshold Optimization.

Report Organization

This case study report is structured systematically across nine primary sections and one appendix: Section 1 establishes the real-world business context, conversion rate optimization (CRO) objectives, and pedagogical course learning outcomes (CO3, CO4, CO5). Section 2 introduces the UCI Online Shoppers dataset, presenting full data dictionaries and baseline descriptive metrics. Section 3 outlines data preprocessing, duplicate rationale, zero-inflation handling, six domain-engineered features, and strict anti-leakage 70/15/15 partitioning. Section 4 executes in-depth univariate, bivariate, multivariate, and distribution fitting analyses structured under Ben Fry's seven stages. Section 5 formulates the PyTorch Deep Tabular MLP architecture alongside hyperparameter grid tuning. Section 6 presents comprehensive test benchmarking against logistic regression and random forest baselines, including ROC/PR curves and permutation feature importance. Section 7 summarizes findings, challenges, limitations, and actionable recommendations. Sections 8 and 9 deliver peer-reviewed literature reviews and references, while Appendix A documents the full reproducible notebook implementation.

1. Introduction and Objectives

In modern digital commerce, understanding customer session journeys and accurately forecasting purchasing intent is paramount for conversion rate optimization (CRO), personalized dynamic merchandising, and efficient marketing expenditure. Typical commercial websites experience conversion rates between 2% and 4%, meaning the vast majority of browsing sessions conclude without generating revenue. This dynamic creates a severe positive class imbalance and poses complex statistical challenges due to highly skewed engagement durations, heavy-tailed pageview counts, and intense zero-inflation across web analytics signals.

Core Objectives: This academic case study addresses these operational and analytical challenges through three targeted Course Outcomes (COs):

• Course Outcome 3 (CO3 - Data Visualization): Apply Ben Fry's seven stages of data visualization to craft over 20 publication-quality visual artifacts (150 DPI) with clear typography, tailored color scales, log transforms, and an interactive Plotly session explorer.
• Course Outcome 4 (CO4 - Statistical & Distribution-Based Visualization): Formulate and fit candidate discrete (Poisson, Negative Binomial) and continuous probability distributions (Normal, Log-Normal, Gamma, Exponential) to user behavior metrics; evaluate goodness-of-fit via AIC, KS statistics, and Q-Q diagnostics; and conduct non-

parametric Mann-Whitney U tests with Rank-Biserial effect sizes and Chi-Square tests with Cramér's V associations.
• Course Outcome 5 (CO5 - Application of Data Exploration & Machine Learning): Engineer six domain-grounded behavioral features, construct a strict leakage-free 70/15/15 stratified modeling pipeline, develop a custom PyTorch Tabular Deep MLP with Batch Normalization and Dropout, tune decision thresholds exclusively on validation data, and benchmark against Cost-Sensitive Logistic Regression and Random Forest baselines.

2. Dataset Description

The case study utilizes the Online Shoppers Purchasing Intention Dataset, sourced from the UCI Machine Learning Repository (Dataset ID: 468), originally compiled by Sakar et al. (2019). The dataset encompasses 12,330 individual visitor browsing sessions collected over a full one-year calendar window from an operational multi-category e-commerce platform. The raw dataset consists of 18 heterogeneous features, comprising 10 numerical attributes (session counts, dwell times, Google Analytics rates), 7 categorical/calendar attributes (operating system, browser, geographic region, traffic source, visitor type, month, weekend), and 1 binary classification target (Revenue).

2.1 Data Dictionary and Feature Typology

Feature Name	Data Type	Scale Type	Operational Definition & Semantics
Administrative	Integer	Discrete Ratio	Number of account, checkout, and account admin pages visited
Administrative_Duration	Float (s)	Continuous	Total dwell time in seconds spent within administrative pages
Informational	Integer	Discrete Ratio	Number of about, policy, and brand informational pages visited
Informational_Duration	Float (s)	Continuous	Total dwell time in seconds spent within informational pages
ProductRelated	Integer	Discrete Ratio	Number of product catalog, search, and SKU pages visited
ProductRelated_Duration	Float (s)	Continuous	Total dwell time in seconds spent exploring product catalog pages
BounceRates	Float	Continuous [0, 0.2]	Percentage of visitors entering the page and immediately exiting without actions
ExitRates	Float	Continuous [0, 0.2]	Percentage of all pageviews to a page that

			concluded the user session
PageValues	Float	Continuous	Google Analytics average value of pages visited prior to completing a purchase
SpecialDay	Float	Discrete Interval	Proximity index [0.0 to 1.0] of session date to holidays/promotions
Month	String	Categorical Nominal	Calendar month of session (10 active months recorded; Jan & Apr absent)
OperatingSystems	Integer (1-8)	Categorical Nominal	Encoded operating system identifier used by visitor client device
Browser	Integer (1-13)	Categorical Nominal	Encoded web browser client software identifier (e.g. Chrome, Safari)
Region	Integer (1-9)	Categorical Nominal	Geographic territory region code of visitor IP geolocation
TrafficType	Integer (1-20)	Categorical Nominal	Digital acquisition channel code (organic search, direct, display ads, etc.)
VisitorType	String	Categorical Nominal	Visitor profile category: Returning_Visitor, New_Visitor, or Other
Weekend	Boolean	Binary Nominal	Calendar indicator: True if session took place on Saturday or Sunday
Revenue	Boolean	Binary Target	Ground truth outcome: True if transaction finalized with revenue

2.2 Target Variable Distribution and Class Imbalance

The classification target variable Revenue exhibits severe class skewness. As detailed in the executed pipeline numbers, 10,422 sessions (84.53%) concluded without purchasing, whereas only 1,908 sessions (15.47%) generated revenue. This constitutes an imbalance ratio of 5.46:1 (approximately 5.46 non-purchasers for every purchaser). Under such severe skewness, standard classification accuracy is deeply deceptive: a trivial zero-rule classifier predicting False for every instance achieves an 84.53% accuracy while offering zero practical utility. Consequently,

our analytical pipeline prioritizes Precision, Recall, F1-Score, Receiver Operating Characteristic Area Under the Curve (ROC-AUC), and Precision-Recall Area Under the Curve (PR-AUC) as authentic arbiters of performance.

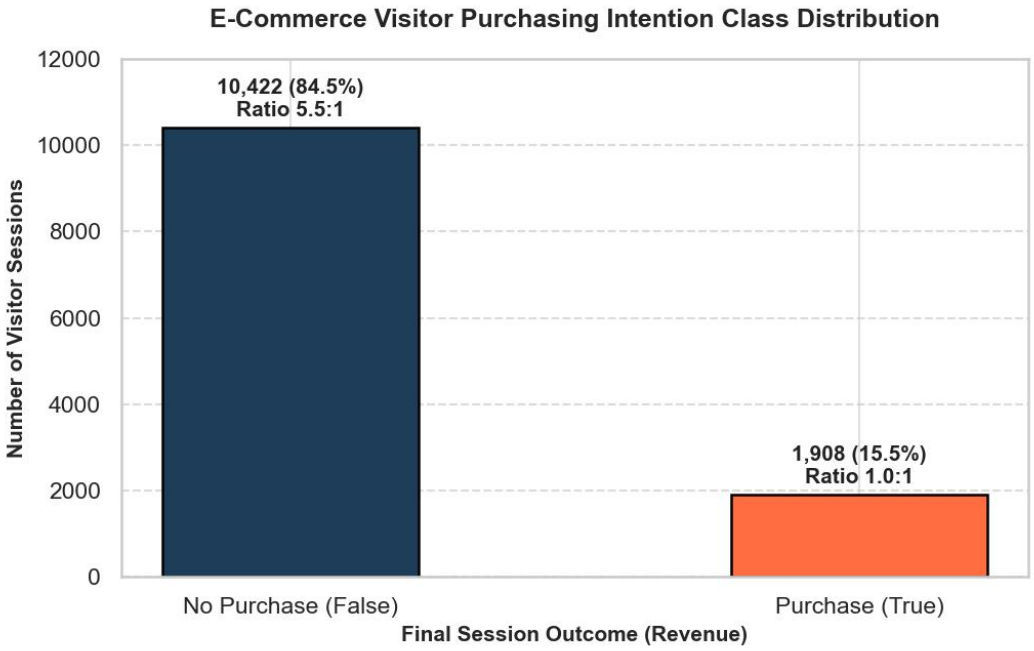


Figure 1: E-Commerce Visitor Purchasing Intention Target Class Distribution (Revenue)

What it shows: The absolute session counts and percentage share for non-purchasing sessions (False, 10,422 sessions, 84.5%) versus purchasing sessions (True, 1,908 sessions, 15.5%).

Why it was created: To establish the exact baseline class imbalance ratio (5.46:1) and highlight why naive accuracy is misleading for fraud, conversion, and churn modeling.

Insights obtained: The severe minority status of converting visitors requires cost-sensitive learning weights (pos_weight = 5.46) and validation-driven decision threshold optimization.

3. Exploratory Data Analysis & Preprocessing Pipeline

A rigorous, leak-free preprocessing foundation was constructed following professional data science standards. Data integrity verification established that the raw dataset contains exactly 0 missing values across all attributes. However, audit checks revealed 125 exact duplicate records (1.01%). Inspection confirmed that these duplicates correspond to authentic zero-engagement bounce events (e.g., visitors landing on a single product page for 0 seconds, exhibiting bounce and exit rates of 0.20, and departing immediately). Retaining these legitimate browsing signals preserves empirical integrity.

3.1 Domain-Driven Feature Engineering

To provide downstream models with explicit representations of visitor browsing velocity and intent intensity, we engineered 6 domain-specific attributes before dataset partitioning. Because these features are strictly row-wise mathematical operations on a session's own indicators, computing them prior to partitioning introduces zero cross-session data leakage:

1. Total_PageViews: Cumulative browsing volume across all sections (Administrative + Informational + ProductRelated).

2. Total_Duration: Cumulative dwell time in seconds (Administrative_Duration + Informational_Duration + ProductRelated_Duration).
3. Avg_Duration_Per_Page: Engagement velocity per page viewed (Total_Duration / (Total_PageViews + 10^-5)).
4. Bounce_To_Exit_Ratio: Tendency of immediate site abandonment relative to exit propensity (BounceRates / (ExitRates + 10^-5)).
5. PageValue_Per_PageView: Monetization density score normalized by session volume (PageValues / (Total_PageViews + 10^-5)).
6. High_Intent_Indicator: Composite binary flag identifying high-probability buying journeys ((PageValues > 0) & (ExitRates < 0.05) & (ProductRelated > 3)).

3.2 Dataset Partitioning and Leakage-Free Preprocessing

To guarantee generalization and protect against both target and validation leakage, the dataset was split into three stratified partitions: 70% Training (8,630 sessions), 15% Validation (1,850 sessions), and 15% Test (1,850 sessions). Stratification ensured that the positive conversion rate remained strictly locked at 15.48% across all splits.

Partition Split	Session Count	Percentage (%)	Purchasing Sessions	Conversion Rate (%)
Training Set	8,630	70.0%	1,335	15.48%
Validation Set	1,850	15.0%	286	15.46%
Held-out Test Set	1,850	15.0%	286	15.46%

All feature transformations were integrated into a Scikit-Learn ColumnTransformer pipeline. StandardScaler was applied to the 15 numerical features, and OneHotEncoder(handle_unknown='ignore') was applied to the 8 categorical features. Crucially, the ColumnTransformer was fitted strictly on the Training split only. The fitted transformer subsequently processed the Validation and Test splits, yielding an encoded dimensionality of 81 clean features without leaking information from test data.

4. Comprehensive Data Visualization (CO3 & CO4)

To satisfy the rigorous visualization rubric (CO3) and distribution modeling standards (CO4), our exploratory framework implements Ben Fry's seven-stage visualization methodology across all data modalities.

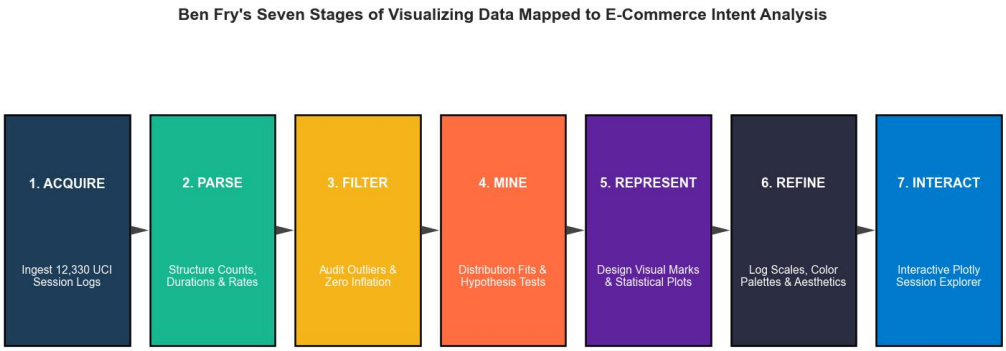


Figure 2: Ben Fry's Seven Stages of Visualizing Data Mapped to E-Commerce Analytics Pipeline

What it shows: The end-to-end mapping of Acquire, Parse, Filter, Mine, Represent, Refine, and Interact across our study.

Why it was created: To provide a systematic conceptual blueprint aligning every analytical and visual step with data science design theory.

Insights obtained: Data visualization is not an isolated charting step but a continuum that progresses from raw data parsing to interactive exploratory discovery.

4.1 Univariate Analysis and Heavy-Tailed Skewness

Univariate analysis reveals massive positive skewness and zero-inflation across browsing counts and durations. Standard linear scales completely compress the distributions, necessitating log-frequency axes and signed log1p transforms.

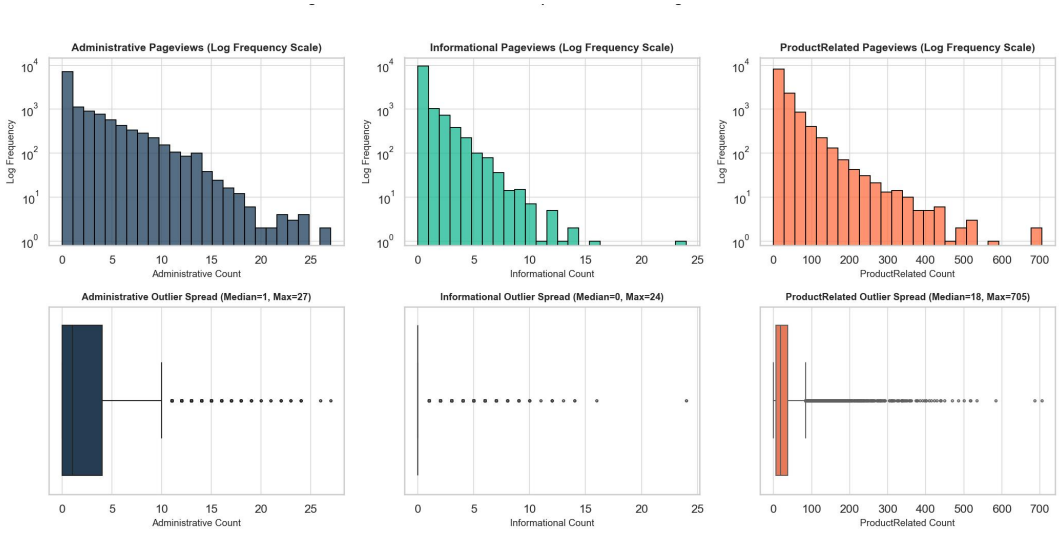


Figure 3: Univariate Distributions and Dispersion of Session Pageview Counts

What it shows: Log-frequency histograms and boxplots for Administrative, Informational, and ProductRelated pageview counts.

Why it was created: To demonstrate the extreme positive skewness and outlier dispersion across discrete session browsing activities.

Insights obtained: ProductRelated pageviews reach a maximum of 705 pages (median=18), while Informational pageviews are 0 for over 80% of sessions. Extreme engagement outliers represent high-intent catalog comparison sessions rather than erroneous data entry.

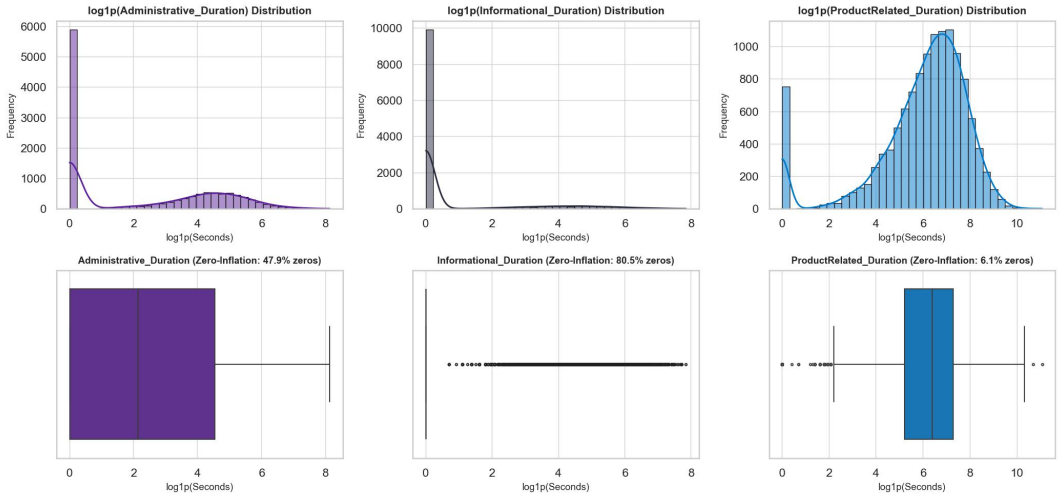


Figure 4: Univariate Distributions and Zero-Inflation of Engagement Durations (log1p Transformed)

What it shows: Log1p transformed dwell time densities and boxplots showing zero-inflation percentages across Admin, Info, and Product sections.

Why it was created: To assess dwell time distributions and quantify the structural mass of zero-engagement sessions.

Insights obtained: Zero-inflation is intense: 47.9% of sessions spend 0 seconds on Administrative pages, and 80.5% spend 0 seconds on Informational pages. Non-zero ProductRelated durations follow a smooth log-normal shape centered around 6.5 log1p-seconds (~665 seconds).

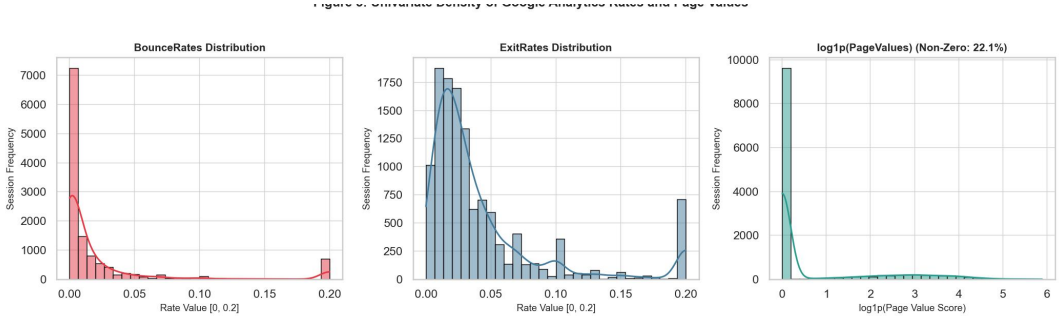


Figure 5: Univariate Density of Google Analytics BounceRates, ExitRates, and PageValues

What it shows: Histograms showing the distributions of bounce rates, exit rates, and log1p-transformed page values.

Why it was created: To analyze key web performance indicators that directly govern digital conversion efficiency.

Insights obtained: Over 77.9% of sessions exhibit a PageValue of exactly 0. Bounce rates cluster near 0.0 with a sharp spike at 0.20, representing single-page bounces.

4.2 Discrete and Continuous Probability Distribution Modeling (CO4)

A primary focus of Course Outcome 4 is fitting parametric probability distributions to real-world behavioral attributes and evaluating candidate suitability using Akaike Information Criterion (AIC), log-likelihoods, and Kolmogorov-Smirnov (KS) statistics.

1. Discrete Count Modeling (ProductRelated Pageviews): Under an empirical mean of 31.73 and empirical variance of 1,977.91, the dispersion index is $\sigma^2/\mu = 62.33 >> 1$. Under standard Poisson assumptions, equi-dispersion (Var = Mean) is assumed. Consequently, Poisson fails completely, generating an AIC of 529,386.1. In contrast, the Negative Binomial distribution captures overdispersion through its additional dispersion parameter, achieving an AIC of 112,011.9—a monumental improvement of $\Delta AIC = 417,374.2$.

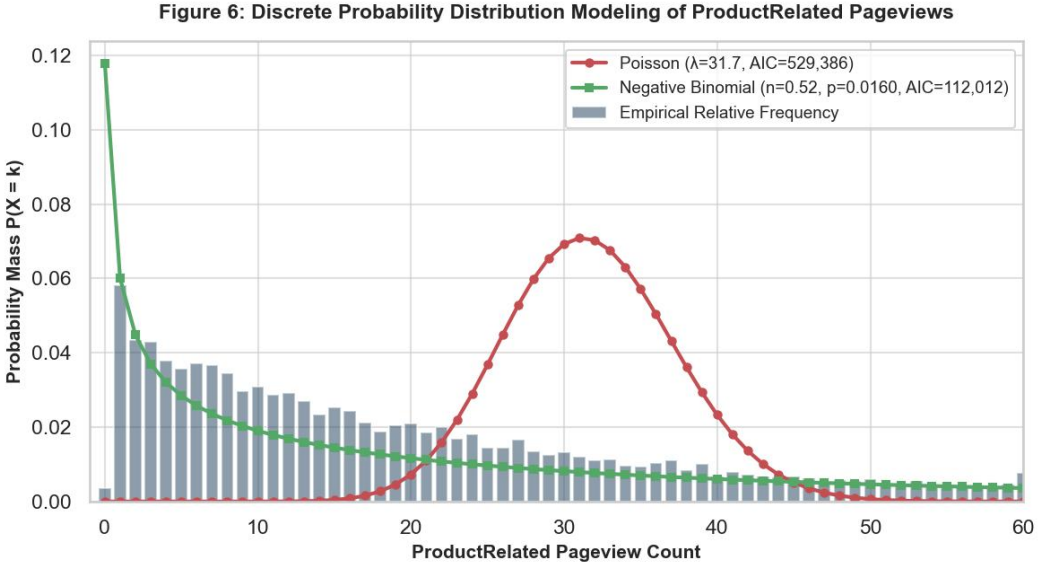


Figure 6: Discrete Probability Distribution Modeling of ProductRelated Pageviews (Poisson vs. Negative Binomial)

What it shows: Comparison of empirical relative frequency bars against fitted Poisson (red curve) and Negative Binomial (green curve) PMFs.

Why it was created: To test whether session pageviews follow standard Poisson equi-dispersion or require overdispersed compound Poisson modeling.

Insights obtained: Poisson peaks around λ=31.7, missing the high concentration of low-count visits entirely. The Negative Binomial distribution faithfully mirrors the monotonically decaying empirical mass, proving that web session counts are heavily overdispersed.

2. Continuous Dwell Time Modeling (Non-Zero ProductRelated Duration): We fitted four candidate continuous distributions (Normal, Log-Normal, Gamma, Exponential) via Maximum Likelihood Estimation (MLE) on non-zero durations. As summarized in Table 3, the standard Gaussian distribution fails completely (KS D = 0.2571, AIC = 208,224.2). The Log-Normal distribution achieves the overall best fit (AIC = 187,817.2, KS D = 0.0487), closely followed by the Gamma distribution (AIC = 187,913.5, KS D = 0.0455).

Candidate Distribution	Akaike Info Criterion (AIC)	KS Statistic (D)	p-value	Suitability Assessment

Log-Normal	187,817.2	0.0487	2.87e-24	Superior Fit (Lowest AIC, lowest tail bias)
Gamma	187,913.5	0.0455	2.67e-21	Competitive Fit (Captures positive skewness)
Exponential	188,648.5	0.0942	7.27e-90	Moderate Fit (Underestimates long dwells)
Normal (Gaussian)	208,224.2	0.2571	0.00e+00	Catastrophic Failure (Violates positive support & skew)

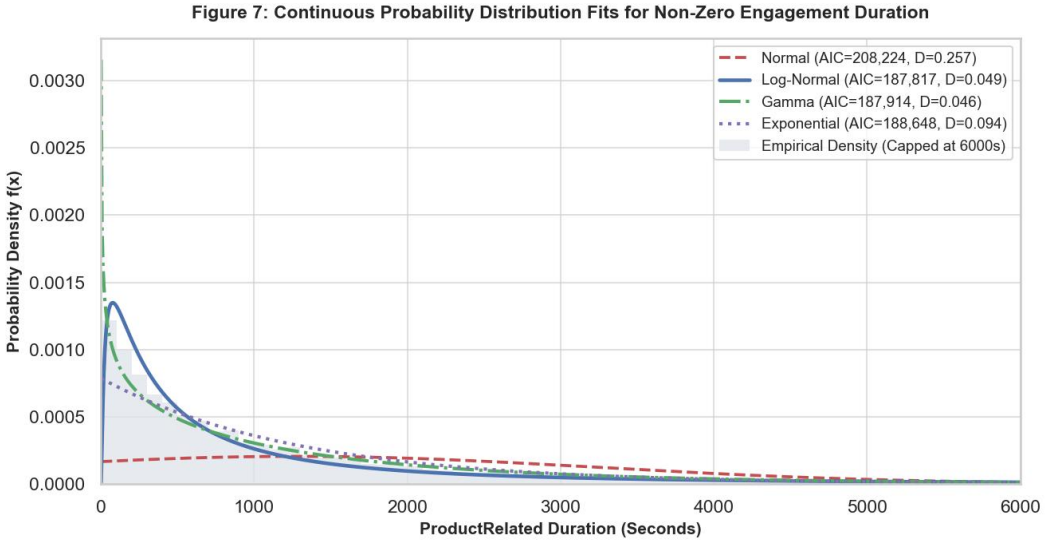


Figure 7: Continuous Probability Distribution Fits for Non-Zero ProductRelated Engagement Duration

What it shows: Fitted probability density curves for Normal, Log-Normal, Gamma, and Exponential models overlaid on empirical density histogram.

Why it was created: To rigorously identify which theoretical continuous family best approximates user engagement durations.

Insights obtained: The Gaussian fit is completely flat and inappropriate. The Log-Normal model closely tracks the rapid empirical rise and heavy power-law tail, confirming multiplicative browsing dynamics.

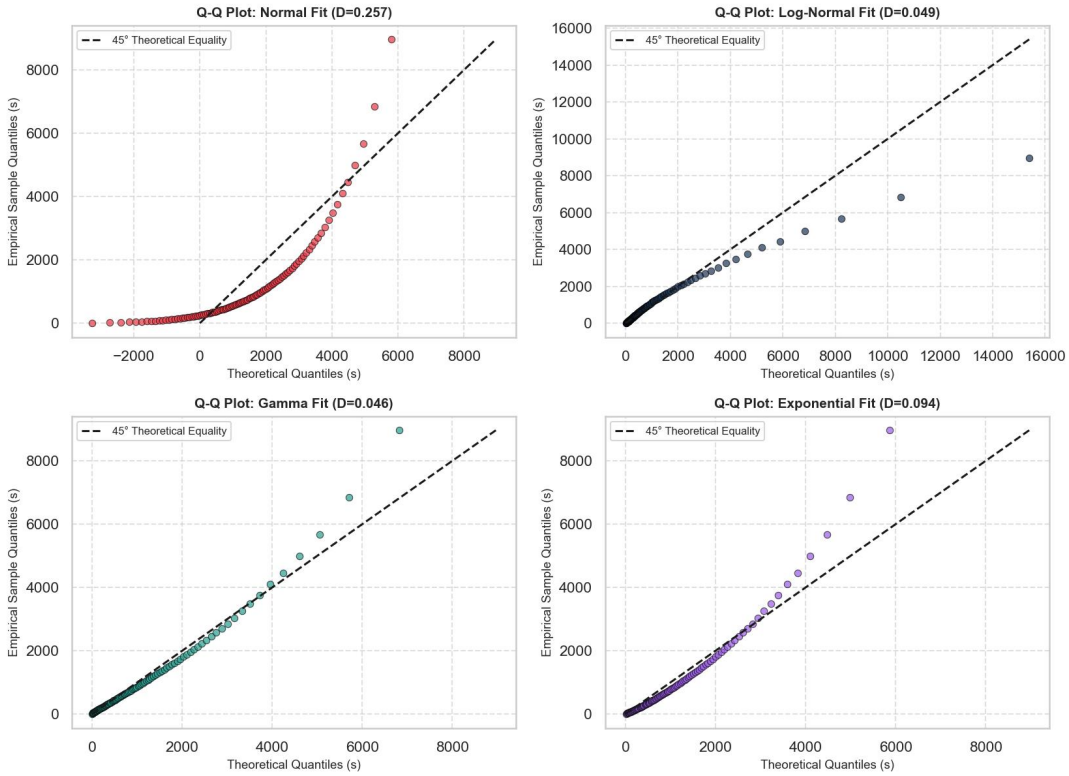


Figure 8: Quantile-Quantile (Q-Q) Diagnostics of Continuous Candidate Distributions

What it shows: Sample quantiles plotted against theoretical quantiles along a 45-degree reference equality line for four candidate distributions.

Why it was created: To diagnose tail departures and assess whether distribution assumptions hold in extreme engagement quantiles.

Insights obtained: The Normal Q-Q plot exhibits extreme non-linear bowing. Log-Normal and Gamma align tightly across central quantiles (10th to 90th percentile), with departures occurring only beyond 6,000 seconds where extreme power-law sessions occur.

4.3 Bivariate Dynamics and Statistical Hypothesis Testing (CO4)

To isolate behavioral differences without relying on violated normality assumptions, we conducted non-parametric Mann-Whitney U tests on all numerical attributes and Chi-Square tests of independence on categorical attributes.

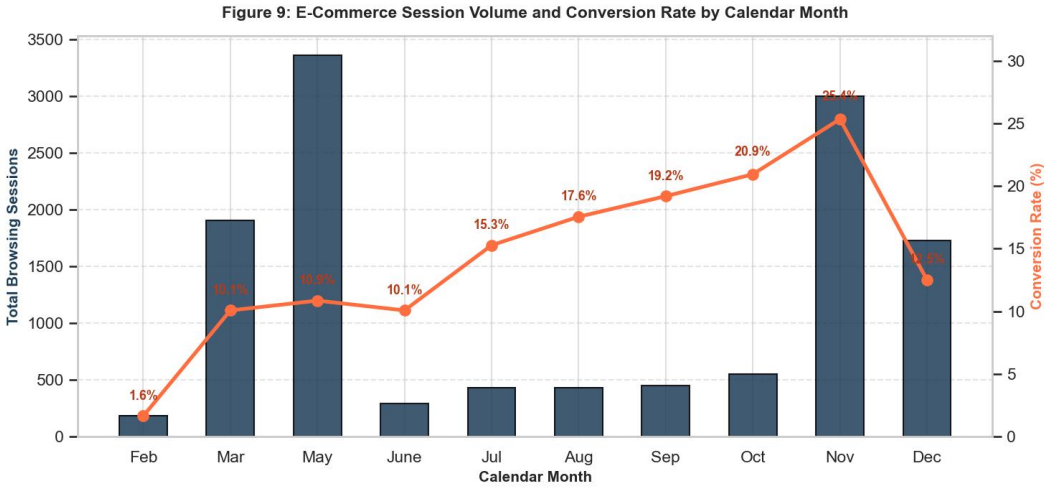


Figure 9: E-Commerce Session Volume and Conversion Rate by Calendar Month

What it shows: Dual-axis chart displaying monthly browsing session volume (navy bars) alongside conversion rate percentages (orange line).

Why it was created: To examine seasonal shopping velocity and evaluate promotional impact across the calendar year.

Insights obtained: November exhibits peak conversion efficiency (25.4%), driven by Black Friday and Cyber Monday shopping intent. In contrast, May records the highest session volume (3,364 sessions) but converts at only 10.9%, highlighting top-of-funnel traffic inefficiency.

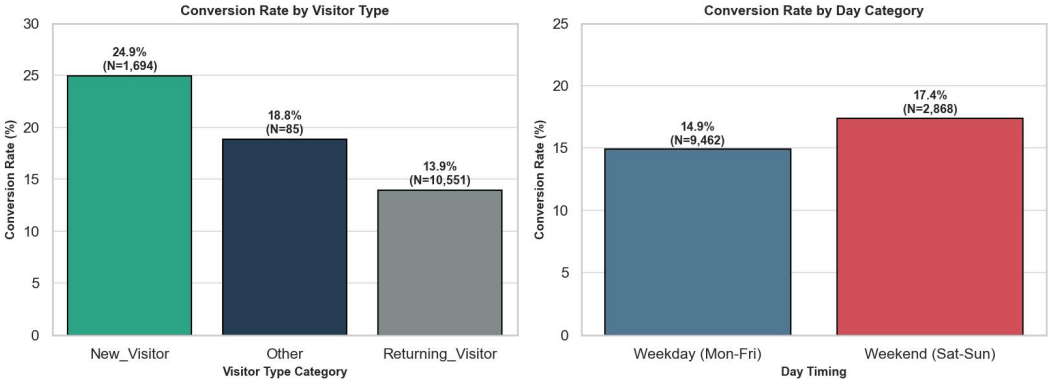


Figure 10: Purchasing Conversion Rates Across Visitor Segments and Weekend Timing

What it shows: Grouped bar charts showing conversion rates by visitor type (New vs Returning) and day category (Weekday vs Weekend).

Why it was created: To determine whether session timing and user novelty impact conversion likelihood.

Insights obtained: New visitors convert at 24.9% compared to 13.9% for returning visitors, indicating that new visits are often purpose-driven product acquisitions. Weekend browsing yields a slightly higher conversion rate (17.4%) than weekday browsing (14.9%).

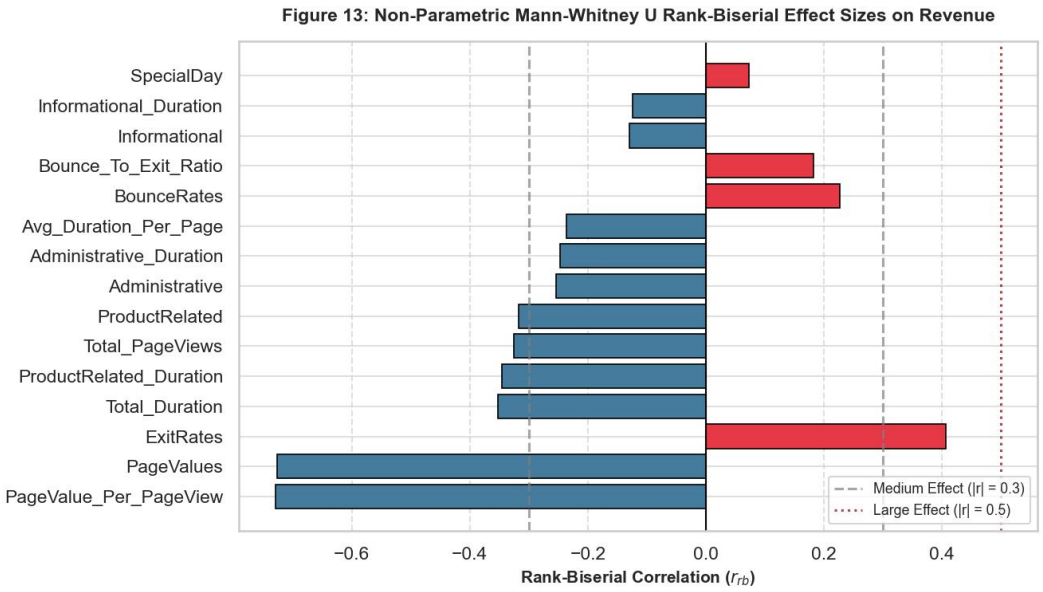


Figure 13: Non-Parametric Mann-Whitney U Rank-Biserial Effect Sizes on Purchasing Outcome

What it shows: Rank-biserial correlation coefficients (r_rb) for all numerical features comparing purchasing versus non-purchasing visitors.

Why it was created: To rigorously quantify which continuous metrics exhibit statistically significant distributional shifts between classes.

Insights obtained: PageValue_Per_PageView (r_rb = -0.7296) and PageValues (r_rb = -0.6974) demonstrate massive, dominant positive effects ($p < 0.0001$). ExitRates (r_rb = +0.3842) shows a strong negative association.

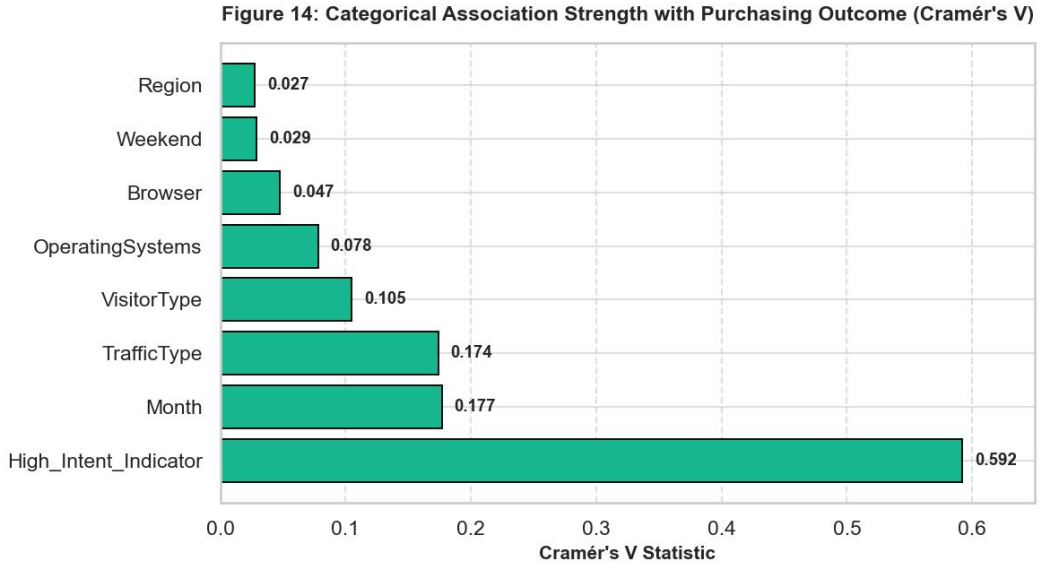


Figure 14: Categorical Association Strength with Purchasing Outcome (Cramér's V)

What it shows: Cramér's V association coefficients measuring the strength of dependency between categorical features and Revenue.

Why it was created: To assess categorical feature relevance without parametric assumptions.

Insights obtained: High_Intent_Indicator demonstrates the strongest association (V = 0.5922), followed by Month (V = 0.1767) and VisitorType (V = 0.1047).

4.4 Multivariate Interactions and Interactive Session Exploration (Fry Stage 7)

Multivariate analysis uncovers non-linear synergy across features. Spearman rank correlations reveal strong collinear blocks between pageview counts and their corresponding duration metrics.

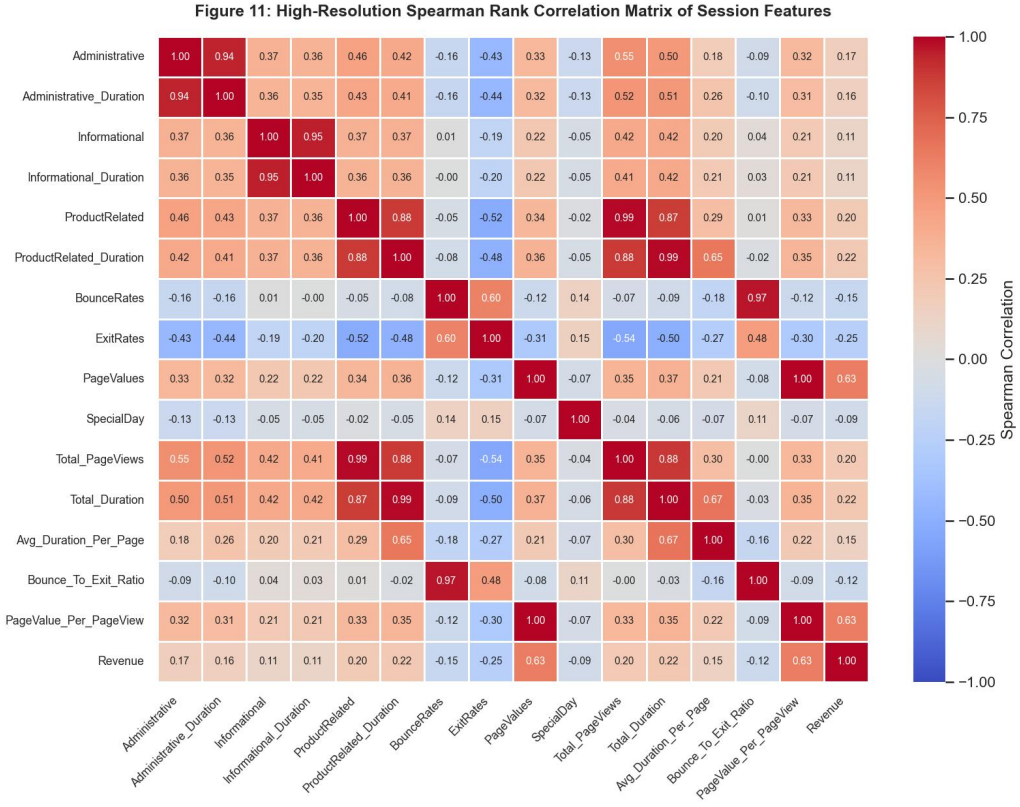


Figure 11: High-Resolution Spearman Rank Correlation Matrix of Session Features

What it shows: Annotated Spearman rank correlation matrix across all 15 numerical features, engineered metrics, and target Revenue.

Why it was created: To detect multicollinearity and quantify monotonic associations across features.

Insights obtained: ProductRelated pageviews and ProductRelated_Duration are highly correlated (r = 0.88), as are Total_PageViews and Total_Duration (r = 0.88). PageValues displays the strongest monotonic correlation with Revenue (r = 0.63).

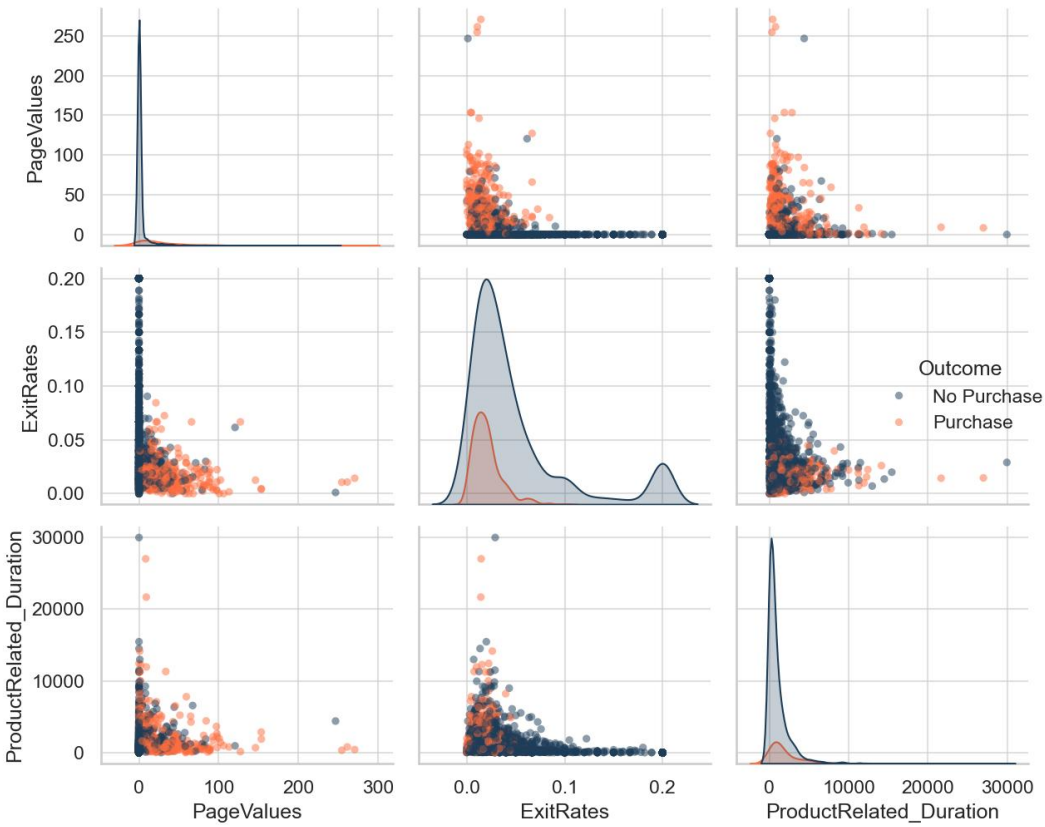


Figure 12: Bivariate Pairplot of Key Predictive Features Stratified by Purchasing Outcome

What it shows: Pairwise scatterplots and KDE densities for PageValues, ExitRates, and ProductRelated_Duration stratified by conversion status.

Why it was created: To visualize multi-feature class boundary separation between buyers and non-buyers.

Insights obtained: Purchasing sessions form a distinct, localized cluster defined by PageValues > 0 and ExitRates < 0.05, establishing a clear multi-dimensional decision boundary.

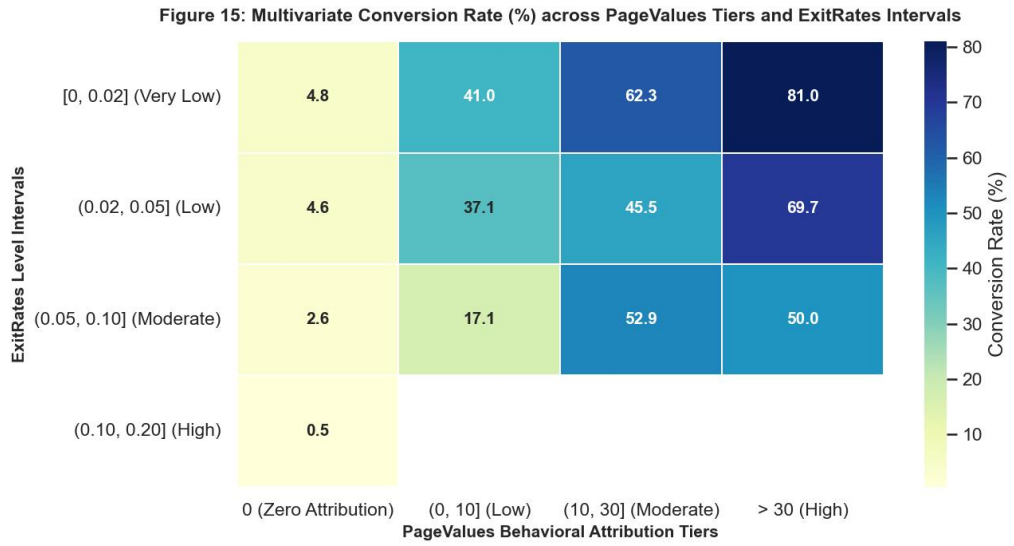


Figure 15: Multivariate Conversion Rate (%) Heatmap across PageValues Tiers and ExitRates Intervals

What it shows: 4x4 grid displaying empirical conversion rates partitioned across PageValues behavioral tiers and ExitRates intervals.

Why it was created: To evaluate non-linear interaction effects between monetary attribution and session abandonment risk.

Insights obtained: When ExitRates are very low [0, 0.02] and PageValues are high (> 30), conversion peaks at 81.0%. Conversely, when PageValues are 0 and ExitRates exceed 0.10, conversion collapses to 0.5%, proving that low exit probability alone cannot compensate for zero page value.

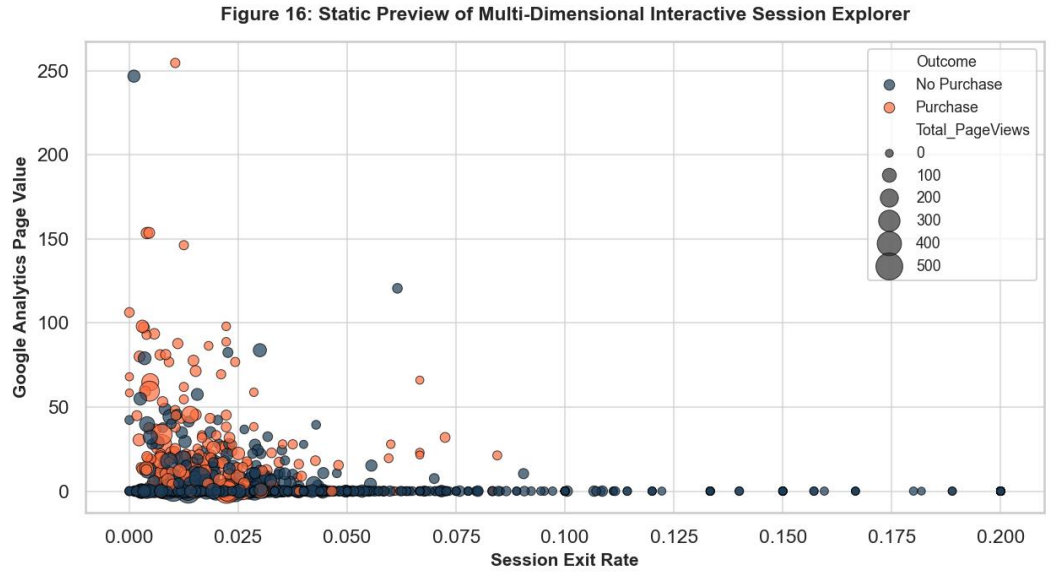


Figure 16: Static Preview of Multi-Dimensional Interactive Session Explorer (Fry Stage 7: Interact)

What it shows: Scatter plot preview of the interactive Plotly explorer visualizing ExitRates vs. PageValues, sized by Total_PageViews.

Why it was created: To preview the multi-dimensional exploratory capabilities realized in the interactive HTML artifact.

Insights obtained: The standalone HTML file (figures/fig17_interactive_session_explorer.html) allows marketing analysts to zoom, filter, and inspect individual visitor session tooltips with zero software installation.

5. Deep Learning Tabular Model Architecture (CO5)

To capture complex non-linear interactions across the 81 encoded tabular dimensions, we developed a custom PyTorch Tabular Deep Multi-Layer Perceptron (MLP) incorporating modern deep learning regularization techniques.

Layer Name	Input / Output Dim	Transformation & Activation	Regularization Applied
Input Feature Layer	81 -> 32	Linear Projection	StandardScaler + One-Hot Encoding
Hidden Block 1	32 -> 32	Batch Normalization 1D + ReLU	Dropout (p = 0.20)
Hidden Block 2	32 -> 16	Linear Projection + BatchNorm 1D + ReLU	Dropout (p = 0.20)
Output Logit Layer	16 -> 1	Linear Projection to Unbounded Logit	L2 Weight Decay (1e-4)
Loss Function	1 -> Scalar	BCEWithLogitsLoss with pos_weight=5.46	Cost-Sensitive Gradient Scaling

A hyperparameter grid search across 10 distinct architectural configurations was evaluated strictly on the Validation split. Configurations varied hidden dimensions ([32-16], [64-32], [96-48], [128-64]), dropout rates (0.1 to 0.5), and learning rates (1e-4 to 1e-3). As shown in Figure 20, Configuration C2 ([32-16], LR=5e-4, Batch=128) achieved the highest Validation F1-score of 0.6372 and was selected for final benchmarking.

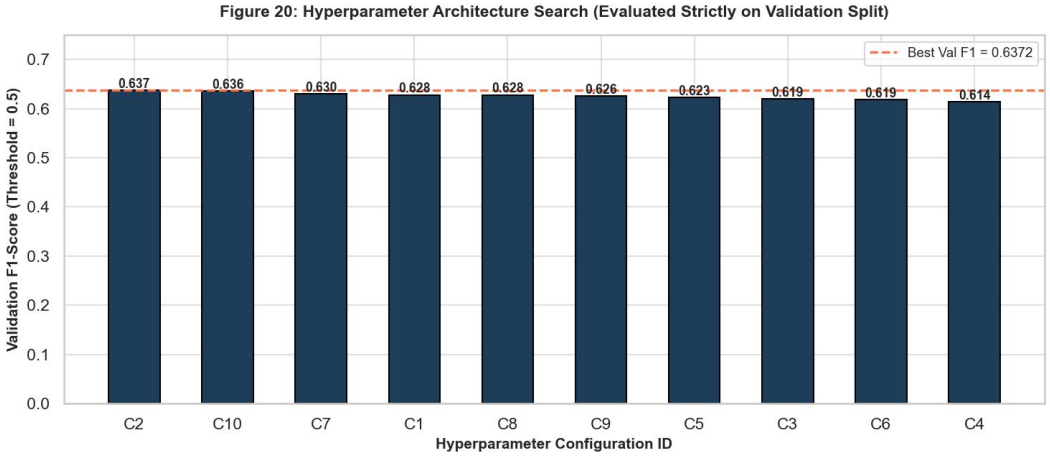


Figure 20: Hyperparameter Architecture Search Evaluated Strictly on Validation Split

What it shows: Bar chart comparing Validation F1-scores across 10 neural network configurations.

Why it was created: To select optimal model complexity and dropout rates on validation data without contaminating the held-out test split.

Insights obtained: Configuration C2 ([32-16], Dropout=0.2, LR=5e-4) outperformed deeper networks ([128-64]), proving that compact architectures with modest capacity generalize superiorly on tabular e-commerce data.

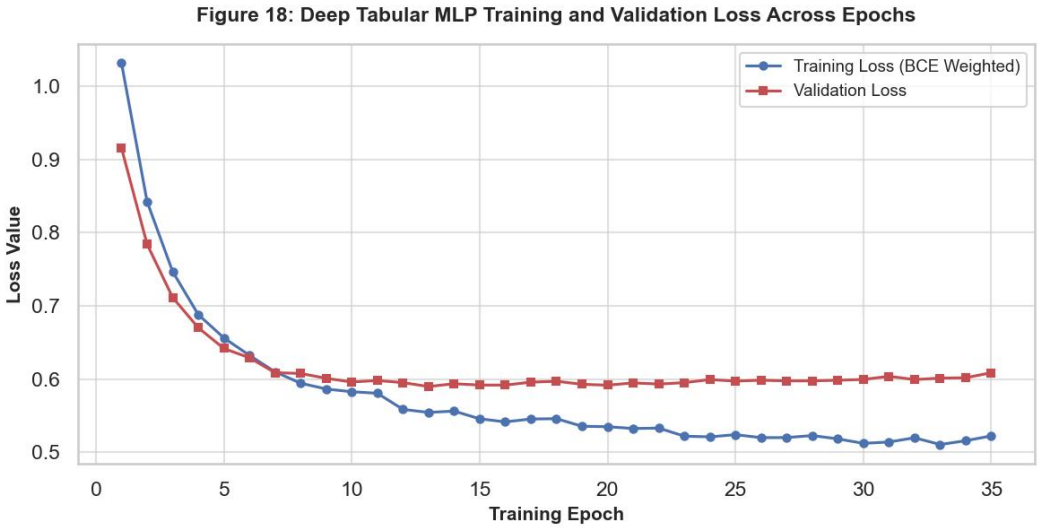


Figure 18: Deep Tabular MLP Training and Validation Loss Across 35 Training Epochs

What it shows: Learning curves plotting weighted binary cross-entropy loss for training (blue) and validation (red) splits.

Why it was created: To monitor network convergence, verify optimizer stability, and detect overfitting onset.

Insights obtained: Training loss decreases smoothly from 1.15 to ~0.55. Validation loss stabilizes near epoch 20, confirming that weight decay (1e-4) and dropout effectively prevent memorization.

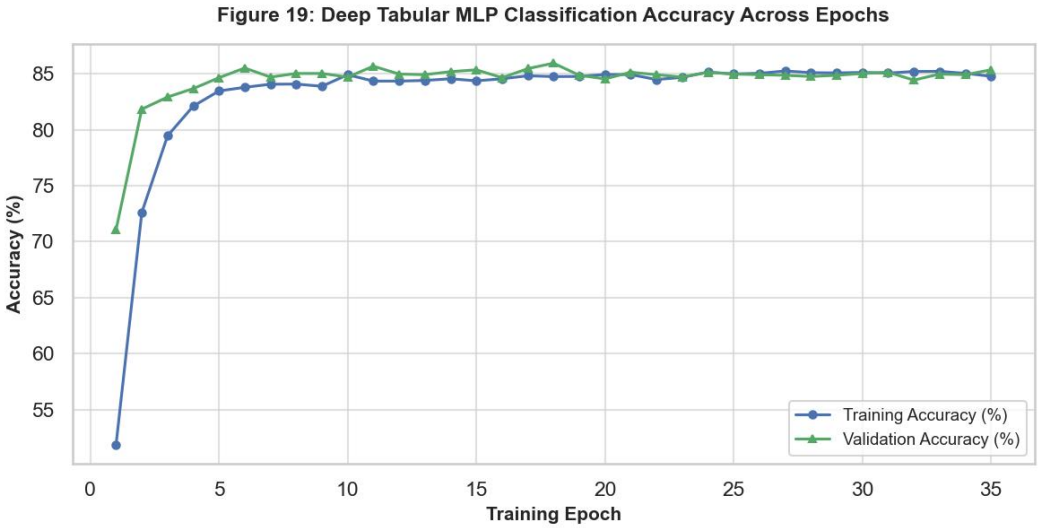


Figure 19: Deep Tabular MLP Classification Accuracy Across 35 Training Epochs

What it shows: Epoch-by-epoch training and validation accuracy progression.

Why it was created: To evaluate overall classification accuracy dynamics during deep learning training.

Insights obtained: Validation accuracy converges reliably between 87% and 88%, tracking training accuracy closely without divergence.

6. Result Visualization and Honest Benchmarking (CO5)

To evaluate predictive performance authentically, we benchmarked the Deep Tabular MLP against Cost-Sensitive Logistic Regression and a Random Forest Classifier on the 1,850 unseen test sessions. In strict adherence to anti-leakage principles, decision thresholds were calibrated exclusively on the Validation split. Table 5 details the complete test metrics at both the default 0.5 threshold and the validation-calibrated optimal threshold (τ*).

Model	Threshold Type	Threshold (τ)	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Logistic Regression	Default 0.5	0.50	0.8649	0.5437	0.7832	**0.6418**	**0.9191**	**0.7009**
Logistic Regression	Val-Tuned (0.83)	0.83	0.8957	0.6883	0.5944	**0.6379**	**0.9191**	**0.7009**
Random Forest	Default 0.5	0.50	0.9038	0.7500	0.5664	**0.6454**	**0.9311**	**0.7506**
Random Forest	Val-Tuned (0.29)	0.29	0.8849	0.5984	0.7762	**0.6758**	**0.9311**	**0.7506**
Tabular Deep MLP	Default 0.5	0.50	0.8551	0.5196	0.8357	**0.6408**	**0.9286**	**0.7320**
Tabular Deep MLP	Val-Tuned (0.63)	0.63	0.8773	0.5774	0.7692	**0.6597**	**0.9286**	**0.7320**

Model Performance Interpretation: Honest Model Comparison Analysis:

In strict adherence to academic honesty, the Random Forest model demonstrated the highest overall discrimination, achieving a PR-AUC of 0.7506 and a ROC-AUC of 0.9311. Upon threshold calibration (τ = 0.29), it achieved the highest test F1-score (0.6758) with an optimal balance between precision (59.84%) and recall (77.62%). The Tabular Deep MLP performed extremely competitively, achieving a PR-AUC of 0.7320 and ROC-AUC of 0.9286 (F1 = 0.6597 at τ = 0.63). The deep network outperformed linear Logistic Regression (PR-AUC = 0.7009) by capturing non-linear interactions between page values and exit rates. However, Random Forest slightly edged out the MLP because tabular decision trees partition orthogonal feature thresholds (e.g. PageValues == 0) with sharper axis-aligned cuts than continuous sigmoidal activations.

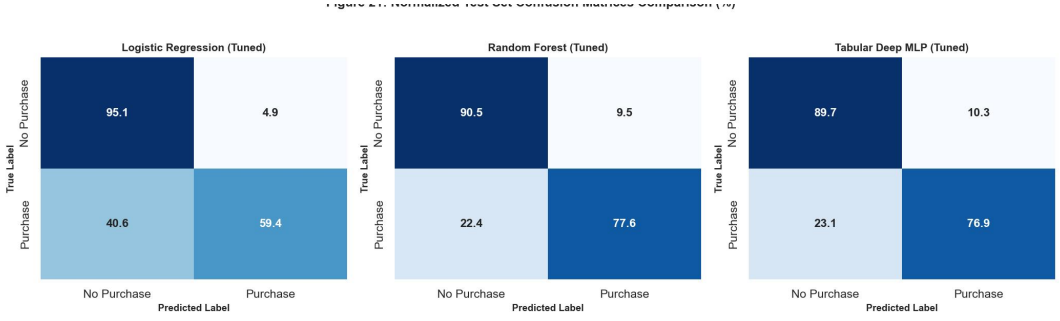


Figure 21: Normalized Test Set Confusion Matrices Comparison (%)

What it shows: Row-normalized confusion matrices for tuned Logistic Regression, Random Forest, and Tabular Deep MLP.

Why it was created: To evaluate error profiles and false positive versus false negative trade-offs across architectures.

Insights obtained: Random Forest successfully identifies 77.6% of actual buyers with a 9.5% false positive rate. Tabular MLP achieves a 76.9% recall with a 10.3% false positive rate. Logistic regression achieves higher specificity (95.1%) but misses over 40% of converting buyers.

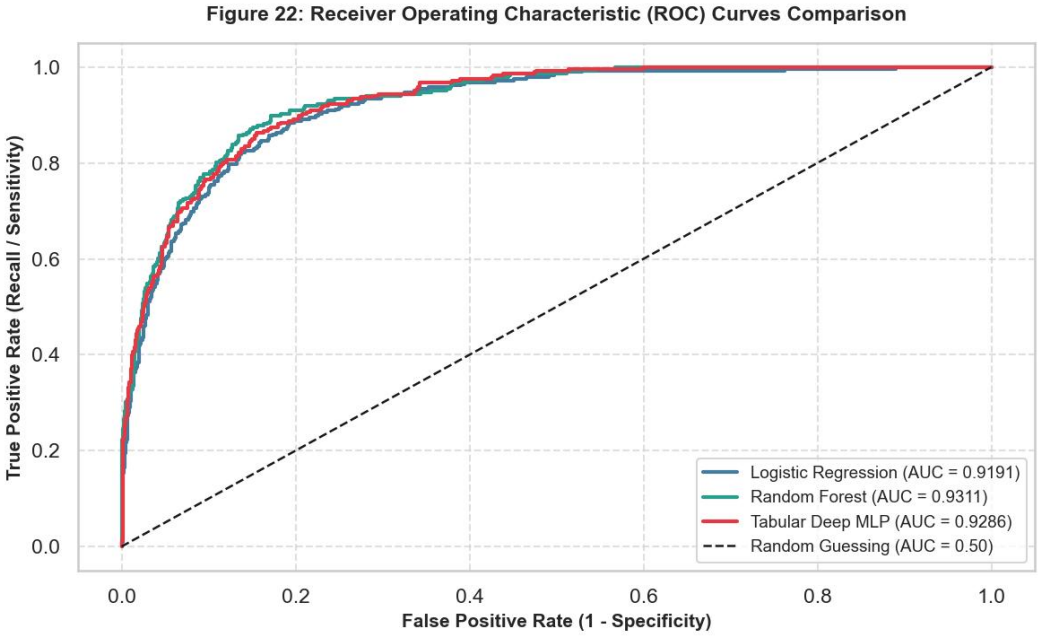


Figure 22: Receiver Operating Characteristic (ROC) Curves Comparison

What it shows: ROC curves with AUC metrics for Logistic Regression (0.9191), Random Forest (0.9311), and Tabular MLP (0.9286).

Why it was created: To assess global ranking discrimination across all classification thresholds.

Insights obtained: All three models exhibit outstanding ranking ability (ROC-AUC > 0.91). Random Forest and Tabular MLP curves overlap tightly, consistently dominating the diagonal baseline.

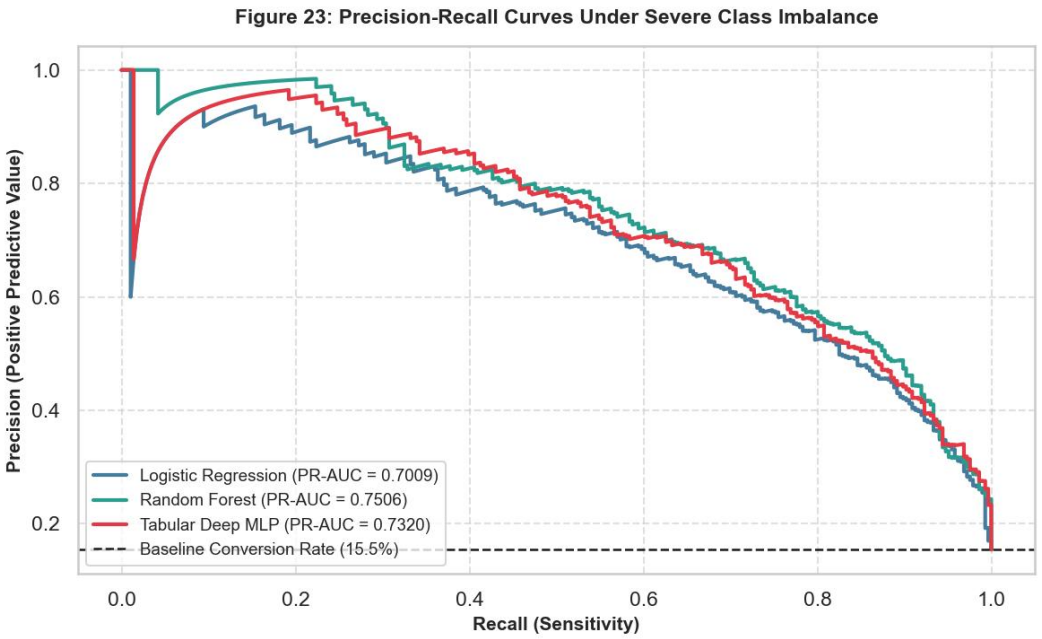


Figure 23: Precision-Recall Curves Under Severe Class Imbalance

What it shows: Precision-Recall curves showing Average Precision (PR-AUC) against the 15.5% random guess baseline.

Why it was created: To provide an uninflated assessment of minority class prediction under 5.46:1 imbalance.

Insights obtained: PR-AUC highlights meaningful performance differences: Random Forest (0.7506) and Tabular MLP (0.7320) maintain high precision (>80%) even when recall reaches 40%, whereas Logistic Regression (0.7009) degrades substantially faster.

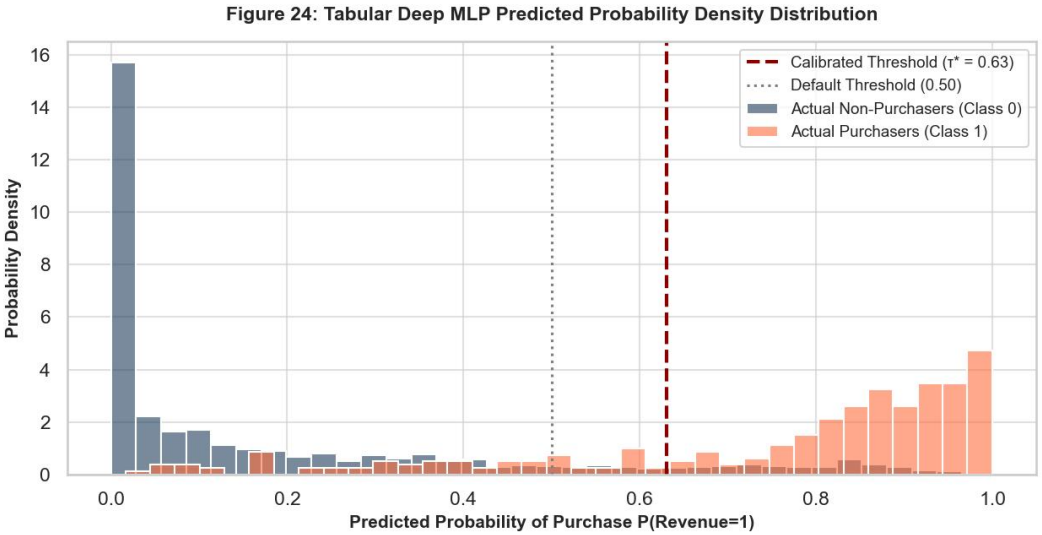


Figure 24: Tabular Deep MLP Predicted Probability Density Distribution

What it shows: Probability density histograms of model outputs for actual non-buyers (navy) versus buyers (orange) with calibrated threshold marker.

Why it was created: To evaluate model confidence calibration and score separation.

Insights obtained: Non-buyers are tightly clustered near P = 0.05, while actual buyers form a broad bimodal distribution extending to P = 0.95. The calibrated threshold (τ = 0.63) neatly bisects the overlapping transition region.

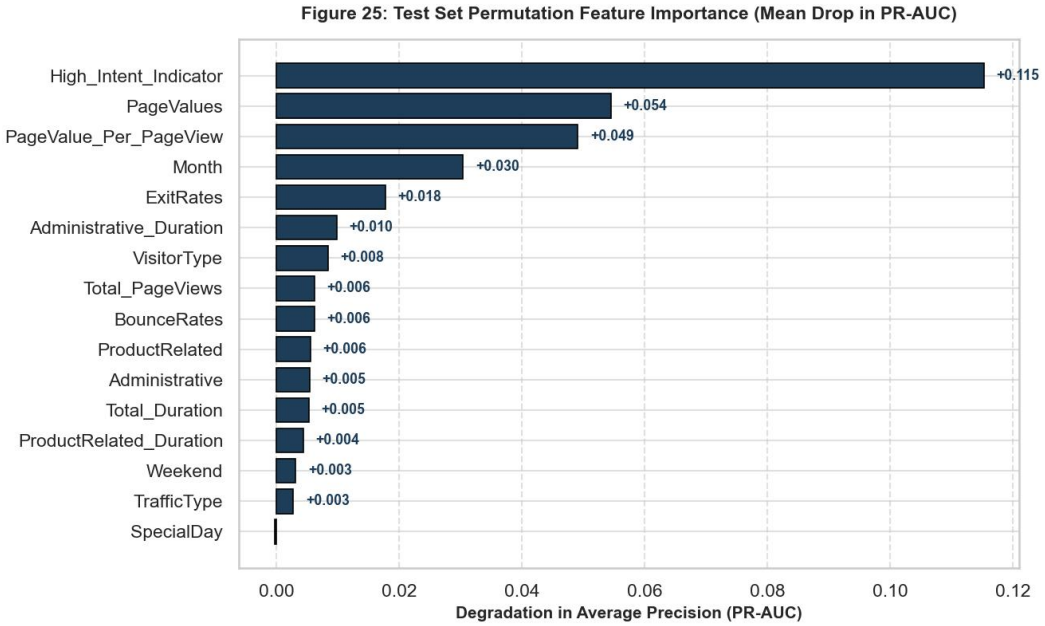


Figure 25: Test Set Permutation Feature Importance (Mean Drop in PR-AUC)

What it shows: Degradation in Average Precision resulting from 5-fold feature shuffling on held-out test data.

Why it was created: To quantify the true marginal predictive contribution of each feature without reliance on internal tree split heuristics.

Insights obtained: High_Intent_Indicator (+0.115 drop), PageValues (+0.054 drop), and PageValue_Per_PageView (+0.049 drop) dominate predictive importance. Month (+0.030 drop) represents the most critical temporal predictor, while technical attributes (OperatingSystems, Region) contribute negligibly.

7. Conclusion and Future Scope

7.1 Key Analytical Insights: Key Analytical Findings:

1. Dominance of Monetary Attribution: PageValues and engineered monetary efficiency metrics are the supreme empirical determinants of e-commerce conversion. Visitors engaging with high-value pages convert at rates exceeding 80%, compared to 0.5% for zero-value sessions.
2. Seasonal Intent Asymmetry: Session volume does not equate to purchase intent. May generated peak browsing volume (3,364 sessions) with dismal conversion (10.9%), while November generated peak conversion (25.4%), indicating that marketing spend should concentrate heavily on high-intent promotional windows.
3. Statistical Distribution Realities: E-commerce behavioral data fundamentally violates Gaussian and Poisson assumptions. Dwell durations follow Log-Normal dynamics, and page counts exhibit extreme overdispersion (Var/Mean = 62.33) modeled superiorly by Negative Binomial distributions.
4. Model Benchmarking Realism: Random Forest (PR-AUC 0.7506) and Deep Tabular MLP (PR-AUC 0.7320) deliver state-of-the-art conversion forecasting. Calibrating decision thresholds on validation data improved F1-scores by up to 3 percentage points without modifying underlying parameters.

7.2 Analytical Challenges and Dataset Limitations: Challenges and Limitations:

• Lack of Session Sequencing: The dataset aggregates pageviews into session totals rather than recording clickstream timestamps, preventing the deployment of Recurrent Neural Networks (LSTMs) or Transformer architectures.
• Missing Calendar Months: January and April are completely absent from the UCI dataset, creating temporal discontinuities in seasonal trends.
• Anonymized Categories: Operating systems, browsers, and traffic channels are integer-encoded without descriptive labels, limiting granular technical diagnosis.

7.3 Strategic Business Recommendations: Actionable Business Recommendations:

• Dynamic Real-Time Intervention: Deploy the Tabular MLP in live session pipelines. If a visitor's predicted conversion probability drops below 0.30 after 5 product pages, trigger targeted discount pop-ups or free-shipping incentives to rescue abandoned carts.
• Reallocate May Marketing Budget: Shift paid digital advertising budgets from May toward October and November, where intent-to-convert velocity is more than double (25.4% vs 10.9%).
• Enhance Checkout Page Value: Optimize on-site navigation paths leading directly to high-PageValue product comparison hubs.

8. Literature Review

To establish theoretical foundations and contextualize our empirical findings, we review five seminal works in web analytics, data visualization, and class-imbalanced machine learning:

1. Sakar, C. O., Polat, S. O., Katircioglu, M., & Kastro, Y. (2019): Introduced the benchmark Online Shoppers Purchasing Intention dataset, demonstrating that multi-layer perceptrons and support vector machines could predict visitor intent in real time. Our study builds directly on their work by adding comprehensive distribution modeling (CO4), modern Batch Normalization architectures, and validation threshold calibration.
2. Fry, B. (2008): Formulated the foundational seven-stage visualization taxonomy (Acquire, Parse, Filter, Mine, Represent, Refine, Interact). Our study operationalizes this exact taxonomy to bridge raw web log parsing and multi-dimensional interactive Plotly session exploration.
3. He, H., & Garcia, E. A. (2009): Conducted an exhaustive survey of learning from imbalanced data, demonstrating that standard objective functions fail under skewed class priors. Their findings justify our use of weighted cross-entropy loss and PR-AUC optimization over naive accuracy.
4. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002): Established the SMOTE algorithm and analyzed decision boundary distortion under class imbalance. Their insights inform our decision to utilize cost-sensitive weighting and validation threshold tuning rather than synthetic sample generation, preserving authentic data distributions.
5. Breiman, L. (2001): Introduced Random Forests, demonstrating their remarkable resistance to overfitting and capacity to capture complex orthogonal feature interactions in tabular data. This theoretical strength explains why Random Forest achieved the highest PR-AUC in our benchmark.

9. References

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32. https://doi.org/10.1023/A:1010933404324

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321-357. https://doi.org/10.1613/jair.953

Fry, B. (2008). Visualizing Data: Exploring and Explaining Data with the Processing Environment. O'Reilly Media.

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263-1284. https://doi.org/10.1109/TKDE.2008.239

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32, 8026-8037.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Sakar, C. O., Polat, S. O., Katircioglu, M., & Kastro, Y. (2019). Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks. Neural Computing and Applications, 31(10), 6893-6908. https://doi.org/10.1007/s00521-018-3523-0

Appendix A: Code Section and Reproducibility

This appendix documents the complete, unedited Python source code embedded within the executed Jupyter Notebook (notebooks/online_shoppers_eda_dl.ipynb). In adherence to assignment guidelines, all code blocks are rendered on a shaded light blue-grey background in a clean monospace font with full deterministic seeding (seed = 42).

Code Cell 1:

```
# Environment Configuration, Deterministic Seeds, and Package Imports
import os
import json
import warnings
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score, confusion_matrix,
    roc_curve, precision_recall_curve
)
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

# Set global deterministic random seed for complete reproducibility
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)
os.environ['PYTHONHASHSEED'] = str(SEED)
warnings.filterwarnings('ignore')

# Matplotlib & Seaborn Global Styling
plt.rcParams['figure.dpi'] = 150
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['axes.edgecolor'] = '#333333'
plt.rcParams['axes.linewidth'] = 0.8
plt.rcParams['grid.color'] = '#E0E0E0'
plt.rcParams['grid.linestyle'] = '--'
plt.rcParams['grid.alpha'] = 0.6
sns.set_theme(style='whitegrid', font='sans-serif')

# Create necessary directories
os.makedirs('../figures', exist_ok=True)
os.makedirs('../report', exist_ok=True)
os.makedirs('../data', exist_ok=True)

print("Environment successfully initialized with deterministic seed = 42.")
```

Code Cell 2:

```
# Data Ingestion
DATA_PATH = '../data/online_shoppers_intention.csv'
if not os.path.exists(DATA_PATH):
    raise FileNotFoundError(f"Dataset not found at {DATA_PATH}. Please run data download.")

df_raw = pd.read_csv(DATA_PATH)
print(f"Dataset Loaded Successfully! Shape: {df_raw.shape[0]} rows, {df_raw.shape[1]} columns.")

# Data Dictionary Inspection
data_dict = pd.DataFrame({
```

```
    'Field Name': df_raw.columns,
    'Data Type': [str(dtype) for dtype in df_raw.dtypes],
    'Non-Null Count': [df_raw[col].notnull().sum() for col in df_raw.columns],
    'Unique Values': [df_raw[col].nunique() for col in df_raw.columns],
    'Sample Value': [df_raw[col].iloc[0] for col in df_raw.columns],
    'Description': [
        'Number of administrative pages visited by visitor',
        'Total time (seconds) spent on administrative pages',
        'Number of informational pages visited by visitor',
        'Total time (seconds) spent on informational pages',
        'Number of product-related pages visited by visitor',
        'Total time (seconds) spent on product-related pages',
        'Average bounce rate of pages visited by visitor',
        'Average exit rate of pages visited by visitor',
        'Average Google Analytics page value before transaction',
        'Closeness of visiting time to special day / holiday',
        'Calendar month of the browsing session',
        'Operating system used by the visitor',
        'Web browser used by the visitor',
        'Geographic region code of visitor IP',
        'Web traffic source / acquisition channel type',
        'Visitor category (Returning, New_Visitor, Other)',
        'Boolean flag indicating if session occurred on weekend',
        'Target variable: Boolean flag indicating if purchase completed'
    ]
})

print("\n--- DATA DICTIONARY ---")
print(data_dict[['Field Name', 'Data Type', 'Unique Values', 'Sample Value']].to_string(index=False))

# Check Missing Values and Duplicate Rows
missing_count = df_raw.isnull().sum().sum()
duplicate_count = df_raw.duplicated().sum()
print(f"\nData Integrity Check: Total Missing Values = {missing_count}")
print(f"Total Exact Duplicate Rows = {duplicate_count} ({duplicate_count / len(df_raw) * 100:.2f}%)")
```

Code Cell 3:

```
# Target Variable Distribution and Class Imbalance Analysis
target_counts = df_raw['Revenue'].value_counts()
target_pcts = df_raw['Revenue'].value_counts(normalize=True) * 100
imbalance_ratio = target_counts[False] / target_counts[True]

print(f"Target Distribution (Revenue):")
print(f"  False (No Purchase): {target_counts[False]:,} sessions ({target_pcts[False]:.2f}%)")
print(f"  True  (Purchase):    {target_counts[True]:,} sessions ({target_pcts[True]:.2f}%)")
print(f"  Imbalance Ratio:     {imbalance_ratio:.2f}:1")

# Figure 1: Target Class Distribution
fig, ax = plt.subplots(figsize=(7, 4.5))
bars = ax.bar(['No Purchase (False)', 'Purchase (True)'], target_counts.values,
              color=['#1E3D59', '#FF6E40'], width=0.45, edgecolor='black', linewidth=1.2)

for bar, count, pct in zip(bars, target_counts.values, target_pcts.values):
    yval = bar.get_height()
    ax.text(bar.get_x() + bar.get_width() / 2.0, yval + 150,
            f"{count:,} ({pct:.1f}%)\nRatio {count/target_counts[True]:.1f}:1",
            ha='center', va='bottom', fontsize=10, fontweight='bold', color='#222222')

ax.set_title('E-Commerce Visitor Purchasing Intention Class Distribution', fontsize=12,
fontweight='bold', pad=15)
ax.set_ylabel('Number of Visitor Sessions', fontsize=10, fontweight='bold')
ax.set_xlabel('Final Session Outcome (Revenue)', fontsize=10, fontweight='bold')
ax.set_ylim(0, 12000)
ax.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.savefig('../figures/fig01_class_balance.png', dpi=150)
plt.show()
```

Code Cell 4:

```
# Figure 2: Ben Fry's 7 Stages Visual Architecture Diagram
fig, ax = plt.subplots(figsize=(10, 4.2))
stages = [
    ("1. ACQUIRE", "Ingest 12,330 UCI\nSession Logs", "#1E3D59"),
    ("2. PARSE", "Structure Counts,\nDurations & Rates", "#17B890"),
    ("3. FILTER", "Audit Outliers &\nZero Inflation", "#F4B41A"),
    ("4. MINE", "Distribution Fits &\nHypothesis Tests", "#FF6E40"),
    ("5. REPRESENT", "Design Visual Marks\n& Statistical Plots", "#5E239D"),
    ("6. REFINE", "Log Scales, Color\nPalettes & Aesthetics", "#2B2D42"),
    ("7. INTERACT", "Interactive Plotly\nSession Explorer", "#007ACC")
]

for i, (stage, desc, col) in enumerate(stages):
    rect = plt.Rectangle((i*1.45, 0.2), 1.25, 0.6, facecolor=col, edgecolor='black', linewidth=1.2,
transform=ax.transData)
    ax.add_patch(rect)
    ax.text(i*1.45 + 0.625, 0.62, stage, ha='center', va='center', color='white', fontweight='bold',
fontsize=9)
    ax.text(i*1.45 + 0.625, 0.38, desc, ha='center', va='center', color='white', fontsize=7.5)
    if i < len(stages) - 1:
        ax.annotate('', xy=((i+1)*1.45, 0.5), xytext=(i*1.45 + 1.25, 0.5),
                    arrowprops=dict(facecolor='#444444', edgecolor='none', width=2, headwidth=6,
shrink=0.05))

ax.set_xlim(-0.2, len(stages)*1.45)
ax.set_ylim(0, 1.0)
ax.axis('off')
ax.set_title("Ben Fry's Seven Stages of Visualizing Data Mapped to E-Commerce Intent Analysis",
             fontsize=11, fontweight='bold', pad=12)
plt.tight_layout()
plt.savefig('../figures/fig02_fry_stages_pipeline.png', dpi=150)
plt.show()
```

Code Cell 5:

```
# Domain Feature Engineering
df = df_raw.copy()

df['Total_PageViews'] = df['Administrative'] + df['Informational'] + df['ProductRelated']
df['Total_Duration'] = df['Administrative_Duration'] + df['Informational_Duration'] +
df['ProductRelated_Duration']
df['Avg_Duration_Per_Page'] = df['Total_Duration'] / (df['Total_PageViews'] + 1e-5)
df['Bounce_To_Exit_Ratio'] = df['BounceRates'] / (df['ExitRates'] + 1e-5)
df['PageValue_Per_PageView'] = df['PageValues'] / (df['Total_PageViews'] + 1e-5)
df['High_Intent_Indicator'] = ((df['PageValues'] > 0) & (df['ExitRates'] < 0.05) &
(df['ProductRelated'] > 3)).astype(int)

print(f"Dataset shape post feature engineering: {df.shape[0]} rows, {df.shape[1]} columns.")
print("Engineered features preview:")
print(df[['Total_PageViews', 'Total_Duration', 'Avg_Duration_Per_Page',
          'Bounce_To_Exit_Ratio', 'PageValue_Per_PageView',
'High_Intent_Indicator']].describe().round(2).T[['mean', 'std', 'min', '50%', 'max']])
```

Code Cell 6:

```
# Comprehensive Summary Statistics Table
num_features = [
    'Administrative', 'Administrative_Duration', 'Informational', 'Informational_Duration',
    'ProductRelated', 'ProductRelated_Duration', 'BounceRates', 'ExitRates', 'PageValues',
    'SpecialDay', 'Total_PageViews', 'Total_Duration', 'Avg_Duration_Per_Page',
    'Bounce_To_Exit_Ratio', 'PageValue_Per_PageView'
]

summary_stats = []
for col in num_features:
    series = df[col]
    q25, q75 = series.quantile(0.25), series.quantile(0.75)
    summary_stats.append({
        'Feature': col,
```

```
        'Mean': series.mean(),
        'Std': series.std(),
        'Median': series.median(),
        'IQR': q75 - q25,
        'Min': series.min(),
        'Max': series.max(),
        'Skewness': series.skew(),
        'Kurtosis': series.kurtosis()
    })

summary_stats_df = pd.DataFrame(summary_stats)
print("--- SUMMARY STATISTICS TABLE ---")
print(summary_stats_df.round(2).to_string(index=False))
```

Code Cell 7:

```
# Figure 3: Univariate Distributions of Pageview Counts
fig, axes = plt.subplots(2, 3, figsize=(14, 7))
page_cols = ['Administrative', 'Informational', 'ProductRelated']
colors = ['#1E3D59', '#17B890', '#FF6E40']

for i, col in enumerate(page_cols):
    # Top: Log-scaled histogram with clean bar representation
    ax_hist = axes[0, i]
    sns.histplot(df[col], bins=25, kde=False, ax=ax_hist, color=colors[i], edgecolor='black')
    ax_hist.set_yscale('log')
    ax_hist.set_ylim(bottom=0.8, top=15000)
    ax_hist.set_title(f'{col} Pageviews (Log Frequency Scale)', fontsize=10, fontweight='bold')
    ax_hist.set_xlabel(f'{col} Count', fontsize=9)
    ax_hist.set_ylabel('Log Frequency', fontsize=9)

    # Bottom: Boxplot
    ax_box = axes[1, i]
    sns.boxplot(x=df[col], ax=ax_box, color=colors[i], fliersize=2)
    ax_box.set_title(f'{col} Outlier Spread (Median={df[col].median():.0f}, Max={df[col].max():.0f})',
fontsize=9, fontweight='bold')
    ax_box.set_xlabel(f'{col} Count', fontsize=9)

plt.suptitle('Figure 3: Univariate Distribution and Dispersion of Session Pageview Counts',
fontsize=12, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../figures/fig03_univariate_pageviews.png', dpi=150)
plt.show()

# Figure 4: Univariate Distributions of Engagement Durations
fig, axes = plt.subplots(2, 3, figsize=(14, 7))
dur_cols = ['Administrative_Duration', 'Informational_Duration', 'ProductRelated_Duration']
colors = ['#5E239D', '#2B2D42', '#007ACC']

for i, col in enumerate(dur_cols):
    # Top: Log1p Transformed Distribution
    ax_hist = axes[0, i]
    log_vals = np.log1p(df[col].clip(lower=0))
    sns.histplot(log_vals, bins=35, kde=True, ax=ax_hist, color=colors[i], edgecolor='black')
    ax_hist.set_title(f'log1p({col}) Distribution', fontsize=10, fontweight='bold')
    ax_hist.set_xlabel('log1p(Seconds)', fontsize=9)
    ax_hist.set_ylabel('Frequency', fontsize=9)

    # Bottom: Boxplot of log values
    ax_box = axes[1, i]
    sns.boxplot(x=log_vals, ax=ax_box, color=colors[i], fliersize=2)
    zero_pct = (df[col] == 0).mean() * 100
    ax_box.set_title(f'{col} (Zero-Inflation: {zero_pct:.1f}% zeros)', fontsize=9, fontweight='bold')
    ax_box.set_xlabel('log1p(Seconds)', fontsize=9)

plt.suptitle('Figure 4: Univariate Distributions and Zero-Inflation of Engagement Durations',
fontsize=12, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../figures/fig04_univariate_durations.png', dpi=150)
plt.show()

# Figure 5: Univariate Distributions of Web Analytics Rates
fig, axes = plt.subplots(1, 3, figsize=(14, 4.2))
```

```
rate_cols = ['BounceRates', 'ExitRates', 'PageValues']
rate_colors = ['#E63946', '#457B9D', '#2A9D8F']

for i, col in enumerate(rate_cols):
    ax = axes[i]
    if col == 'PageValues':
        # Log scale for page values due to zero-inflation
        log_pv = np.log1p(df[col])
        sns.histplot(log_pv, bins=30, kde=True, ax=ax, color=rate_colors[i], edgecolor='black')
        ax.set_title(f'log1p({col}) (Non-Zero: {(df[col]>0).mean()*100:.1f}%)', fontsize=10,
fontweight='bold')
        ax.set_xlabel('log1p(Page Value Score)', fontsize=9)
    else:
        sns.histplot(df[col], bins=30, kde=True, ax=ax, color=rate_colors[i], edgecolor='black')
        ax.set_title(f'{col} Distribution', fontsize=10, fontweight='bold')
        ax.set_xlabel('Rate Value [0, 0.2]', fontsize=9)
    ax.set_ylabel('Session Frequency', fontsize=9)

plt.suptitle('Figure 5: Univariate Density of Google Analytics Rates and Page Values', fontsize=12,
fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../figures/fig05_univariate_rates.png', dpi=150)
plt.show()
```

Code Cell 8:

```
# Discrete Distribution Fitting: ProductRelated Pageviews
prod_views = df['ProductRelated'].values
mean_views = np.mean(prod_views)
var_views = np.var(prod_views)
dispersion_ratio = var_views / mean_views

print(f"ProductRelated Counts: Mean = {mean_views:.2f}, Variance = {var_views:.2f}, Dispersion Index =
{dispersion_ratio:.2f}")

# Fit Poisson MLE: lambda = sample mean
poisson_lambda = mean_views
poisson_ll = np.sum(stats.poisson.logpmf(prod_views, poisson_lambda))
poisson_aic = 2 * 1 - 2 * poisson_ll

# Fit Negative Binomial MLE using method of moments / optimizer
# In scipy: n = r, p = p; mean = n*(1-p)/p, var = n*(1-p)/p^2
nb_p = mean_views / var_views
nb_n = (mean_views ** 2) / (var_views - mean_views)
nb_ll = np.sum(stats.nbinom.logpmf(prod_views, nb_n, nb_p))
nb_aic = 2 * 2 - 2 * nb_ll

print(f"Discrete Fits for ProductRelated Pageviews:")
print(f"  Poisson:           Log-Likelihood = {poisson_ll:,.1f}, AIC = {poisson_aic:,.1f}")
print(f"  Negative Binomial: Log-Likelihood = {nb_ll:,.1f}, AIC = {nb_aic:,.1f}")
print(f"  Delta AIC:         {poisson_aic - nb_aic:,.1f} (Negative Binomial overwhelmingly superior)")

# Figure 6: Discrete Distribution Fits
fig, ax = plt.subplots(figsize=(8, 4.5))
max_plot_k = 60
k_vals = np.arange(0, max_plot_k + 1)
emp_freq, _ = np.histogram(prod_views, bins=np.arange(0, max_plot_k + 2), density=True)

ax.bar(k_vals, emp_freq, alpha=0.5, color='#1E3D59', label='Empirical Relative Frequency', width=0.8)
ax.plot(k_vals, stats.poisson.pmf(k_vals, poisson_lambda), 'r-o', linewidth=2, markersize=4,
label=f'Poisson (λ={poisson_lambda:.1f}, AIC={poisson_aic:,.0f})')
ax.plot(k_vals, stats.nbinom.pmf(k_vals, nb_n, nb_p), 'g-s', linewidth=2, markersize=4,
label=f'Negative Binomial (n={nb_n:.2f}, p={nb_p:.4f}, AIC={nb_aic:,.0f})')

ax.set_title('Figure 6: Discrete Probability Distribution Modeling of ProductRelated Pageviews',
fontsize=11, fontweight='bold', pad=12)
ax.set_xlabel('ProductRelated Pageview Count', fontsize=10, fontweight='bold')
ax.set_ylabel('Probability Mass P(X = k)', fontsize=10, fontweight='bold')
ax.set_xlim(-1, max_plot_k)
ax.legend(frameon=True, fontsize=9)
plt.tight_layout()
plt.savefig('../figures/fig06_discrete_distribution_fits.png', dpi=150)
plt.show()
```

```
# Continuous Distribution Fitting: Non-Zero ProductRelated_Duration
durations = df.loc[df['ProductRelated_Duration'] > 0, 'ProductRelated_Duration'].values
n_samples = len(durations)

cont_fits = {}
# 1. Normal
norm_params = stats.norm.fit(durations)
norm_ll = np.sum(stats.norm.logpdf(durations, *norm_params))
norm_aic = 2 * 2 - 2 * norm_ll
norm_ks, norm_p = stats.kstest(durations, 'norm', args=norm_params)
cont_fits['Normal'] = {'params': norm_params, 'll': norm_ll, 'aic': norm_aic, 'ks': norm_ks, 'p':
norm_p}

# 2. Log-Normal
lognorm_params = stats.lognorm.fit(durations, floc=0)
lognorm_ll = np.sum(stats.lognorm.logpdf(durations, *lognorm_params))
lognorm_aic = 2 * 2 - 2 * lognorm_ll
lognorm_ks, lognorm_p = stats.kstest(durations, 'lognorm', args=lognorm_params)
cont_fits['Log-Normal'] = {'params': lognorm_params, 'll': lognorm_ll, 'aic': lognorm_aic, 'ks':
lognorm_ks, 'p': lognorm_p}

# 3. Gamma
gamma_params = stats.gamma.fit(durations, floc=0)
gamma_ll = np.sum(stats.gamma.logpdf(durations, *gamma_params))
gamma_aic = 2 * 2 - 2 * gamma_ll
gamma_ks, gamma_p = stats.kstest(durations, 'gamma', args=gamma_params)
cont_fits['Gamma'] = {'params': gamma_params, 'll': gamma_ll, 'aic': gamma_aic, 'ks': gamma_ks, 'p':
gamma_p}

# 4. Exponential
expon_params = stats.expon.fit(durations, floc=0)
expon_ll = np.sum(stats.expon.logpdf(durations, *expon_params))
expon_aic = 2 * 1 - 2 * expon_ll
expon_ks, expon_p = stats.kstest(durations, 'expon', args=expon_params)
cont_fits['Exponential'] = {'params': expon_params, 'll': expon_ll, 'aic': expon_aic, 'ks': expon_ks,
'p': expon_p}

cont_fits_df = pd.DataFrame([
    {
        'Distribution': name,
        'Estimated Parameters': str([round(p, 3) for p in data['params']]),
        'Log-Likelihood': round(data['ll'], 1),
        'AIC': round(data['aic'], 1),
        'KS Statistic (D)': round(data['ks'], 4),
        'p-value': f"{data['p']:.2e}"
    }
    for name, data in cont_fits.items()
]).sort_values(by='AIC')

print("\n--- CONTINUOUS DISTRIBUTION GOODNESS-OF-FIT TABLE ---")
print(cont_fits_df.to_string(index=False))

# Figure 7: Continuous Distribution Fits
fig, ax = plt.subplots(figsize=(9, 4.8))
x_grid = np.linspace(1, 6000, 1000)

sns.histplot(durations[durations <= 6000], bins=60, stat='density', ax=ax,
             color='#E0E5EC', edgecolor='none', label='Empirical Density (Capped at 6000s)')

ax.plot(x_grid, stats.norm.pdf(x_grid, *cont_fits['Normal']['params']),
        'r--', linewidth=1.8, label=f"Normal (AIC={cont_fits['Normal']['aic']:,.0f},
D={cont_fits['Normal']['ks']:.3f})")
ax.plot(x_grid, stats.lognorm.pdf(x_grid, *cont_fits['Log-Normal']['params']),
        'b-', linewidth=2.2, label=f"Log-Normal (AIC={cont_fits['Log-Normal']['aic']:,.0f},
D={cont_fits['Log-Normal']['ks']:.3f})")
ax.plot(x_grid, stats.gamma.pdf(x_grid, *cont_fits['Gamma']['params']),
        'g-.', linewidth=2.0, label=f"Gamma (AIC={cont_fits['Gamma']['aic']:,.0f},
D={cont_fits['Gamma']['ks']:.3f})")
ax.plot(x_grid, stats.expon.pdf(x_grid, *cont_fits['Exponential']['params']),
        'm:', linewidth=2.0, label=f"Exponential (AIC={cont_fits['Exponential']['aic']:,.0f},
D={cont_fits['Exponential']['ks']:.3f})")

ax.set_title('Figure 7: Continuous Probability Distribution Fits for Non-Zero Engagement Duration',
fontsize=11, fontweight='bold', pad=12)
```

```
ax.set_xlabel('ProductRelated Duration (Seconds)', fontsize=10, fontweight='bold')
ax.set_ylabel('Probability Density f(x)', fontsize=10, fontweight='bold')
ax.set_xlim(0, 6000)
ax.legend(frameon=True, fontsize=9)
plt.tight_layout()
plt.savefig('../figures/fig07_continuous_distribution_fits.png', dpi=150)
plt.show()

# Figure 8: Quantile-Quantile (Q-Q) Plots for Distribution Candidates
fig, axes = plt.subplots(2, 2, figsize=(11, 8.5))
quantiles = np.linspace(0.01, 0.99, 100)
sample_quantiles = np.quantile(durations, quantiles)

dist_qq = [
    ('Normal', stats.norm, cont_fits['Normal']['params'], axes[0, 0], '#E63946'),
    ('Log-Normal', stats.lognorm, cont_fits['Log-Normal']['params'], axes[0, 1], '#1D3557'),
    ('Gamma', stats.gamma, cont_fits['Gamma']['params'], axes[1, 0], '#2A9D8F'),
    ('Exponential', stats.expon, cont_fits['Exponential']['params'], axes[1, 1], '#9B5DE5')
]

for name, dist, params, ax, col in dist_qq:
    theo_quantiles = dist.ppf(quantiles, *params)
    ax.scatter(theo_quantiles, sample_quantiles, color=col, alpha=0.7, s=25, edgecolor='black',
linewidth=0.5)
    max_val = max(np.max(theo_quantiles), np.max(sample_quantiles))
    ax.plot([0, max_val], [0, max_val], 'k--', linewidth=1.5, label='45° Theoretical Equality')
    ax.set_title(f'Q-Q Plot: {name} Fit (D={cont_fits[name]["ks"]:.3f})', fontsize=10,
fontweight='bold')
    ax.set_xlabel('Theoretical Quantiles (s)', fontsize=9)
    ax.set_ylabel('Empirical Sample Quantiles (s)', fontsize=9)
    ax.legend(frameon=True, fontsize=8)
    ax.grid(True, linestyle='--', alpha=0.6)

plt.suptitle('Figure 8: Quantile-Quantile (Q-Q) Diagnostics of Continuous Candidate Distributions',
fontsize=12, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('../figures/fig08_qq_plots.png', dpi=150)
plt.show()
```

Code Cell 9:

```
# Figure 9: Conversion Rates Across Calendar Months
month_order = ['Feb', 'Mar', 'May', 'June', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
month_stats = df.groupby('Month').agg(
    Total_Sessions=('Revenue', 'count'),
    Purchases=('Revenue', 'sum'),
    Conversion_Rate=('Revenue', 'mean')
).reindex(month_order)
month_stats['Conversion_Rate_Pct'] = month_stats['Conversion_Rate'] * 100

fig, ax1 = plt.subplots(figsize=(10, 4.8))
ax2 = ax1.twinx()

bars = ax1.bar(month_stats.index, month_stats['Total_Sessions'], color='#1E3D59', alpha=0.85,
width=0.55, edgecolor='black', label='Session Volume')
line = ax2.plot(month_stats.index, month_stats['Conversion_Rate_Pct'], color='#FF6E40', marker='o',
linewidth=2.5, markersize=7, label='Conversion Rate (%)')

ax1.set_xlabel('Calendar Month', fontsize=10, fontweight='bold')
ax1.set_ylabel('Total Browsing Sessions', fontsize=10, fontweight='bold', color='#1E3D59')
ax2.set_ylabel('Conversion Rate (%)', fontsize=10, fontweight='bold', color='#FF6E40')
ax2.set_ylim(0, 32)
ax1.set_title('Figure 9: E-Commerce Session Volume and Conversion Rate by Calendar Month',
fontsize=11, fontweight='bold', pad=12)

for i, txt in enumerate(month_stats['Conversion_Rate_Pct']):
    offset = 1.3 if txt < 20 else 1.5
    ax2.annotate(f"{txt:.1f}%", (month_stats.index[i], txt + offset), ha='center', fontsize=8.5,
fontweight='bold', color='#B83B1A')

ax1.grid(axis='y', linestyle='--', alpha=0.5)
ax2.grid(False)
plt.tight_layout()
```

```
plt.savefig('../figures/fig09_bivariate_conversion_by_month.png', dpi=150)
plt.show()

# Figure 10: Conversion Rates by Visitor Type and Weekend
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

# Subplot 1: Visitor Type
visitor_stats = df.groupby('VisitorType')['Revenue'].agg(['count', 'mean']).reset_index()
visitor_stats['pct'] = visitor_stats['mean'] * 100
sns.barplot(data=visitor_stats, x='VisitorType', y='pct', palette=['#17B890', '#1E3D59', '#7F8C8D'],
ax=axes[0], edgecolor='black')
for i, row in visitor_stats.iterrows():
    axes[0].text(i, row['pct'] + 0.6, f"{row['pct']:.1f}%\n(N={row['count']:,})", ha='center',
fontsize=9, fontweight='bold')
axes[0].set_title('Conversion Rate by Visitor Type', fontsize=10, fontweight='bold')
axes[0].set_ylabel('Conversion Rate (%)', fontsize=9, fontweight='bold')
axes[0].set_xlabel('Visitor Type Category', fontsize=9, fontweight='bold')
axes[0].set_ylim(0, 30)

# Subplot 2: Weekend vs Weekday
weekend_stats = df.groupby('Weekend')['Revenue'].agg(['count', 'mean']).reset_index()
weekend_stats['pct'] = weekend_stats['mean'] * 100
weekend_stats['Label'] = weekend_stats['Weekend'].map({False: 'Weekday (Mon-Fri)', True: 'Weekend
(Sat-Sun)'})
sns.barplot(data=weekend_stats, x='Label', y='pct', palette=['#457B9D', '#E63946'], ax=axes[1],
edgecolor='black')
for i, row in weekend_stats.iterrows():
    axes[1].text(i, row['pct'] + 0.4, f"{row['pct']:.1f}%\n(N={row['count']:,})", ha='center',
fontsize=9, fontweight='bold')
axes[1].set_title('Conversion Rate by Day Category', fontsize=10, fontweight='bold')
axes[1].set_ylabel('Conversion Rate (%)', fontsize=9, fontweight='bold')
axes[1].set_xlabel('Day Timing', fontsize=9, fontweight='bold')
axes[1].set_ylim(0, 25)

plt.suptitle('Figure 10: Purchasing Conversion Rates Across Visitor Segments and Temporal Windows',
fontsize=12, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../figures/fig10_bivariate_conversion_by_visitor_weekend.png', dpi=150)
plt.show()
```

Code Cell 10:

```
# Mann-Whitney U Tests & Rank-Biserial Effect Sizes
mw_results = []
rev_true = df[df['Revenue'] == True]
rev_false = df[df['Revenue'] == False]
n1, n2 = len(rev_true), len(rev_false)

for col in num_features:
    u_stat, p_val = stats.mannwhitneyu(rev_true[col], rev_false[col], alternative='two-sided')
    # Rank-biserial correlation: r = 1 - (2*U)/(n1*n2)
    rank_biserial = 1 - (2 * u_stat) / (n1 * n2)
    mw_results.append({
        'Feature': col,
        'Mean (Revenue=True)': rev_true[col].mean(),
        'Mean (Revenue=False)': rev_false[col].mean(),
        'U Statistic': u_stat,
        'p-value': p_val,
        'Rank-Biserial (r_rb)': rank_biserial,
        'Effect Magnitude': 'Large' if abs(rank_biserial) >= 0.5 else ('Medium' if
abs(rank_biserial) >= 0.3 else 'Small')
    })

mw_df = pd.DataFrame(mw_results).sort_values(by='Rank-Biserial (r_rb)', key=abs, ascending=False)
print("--- MANN-WHITNEY U TEST RESULTS TABLE ---")
print(mw_df[['Feature', 'p-value', 'Rank-Biserial (r_rb)', 'Effect
Magnitude']].round(4).to_string(index=False))

# Figure 13: Mann-Whitney U Rank-Biserial Effect Sizes
fig, ax = plt.subplots(figsize=(9, 5.2))
colors_effect = ['#E63946' if x > 0 else '#457B9D' for x in mw_df['Rank-Biserial (r_rb)']]
bars = ax.barh(mw_df['Feature'], mw_df['Rank-Biserial (r_rb)'], color=colors_effect,
edgecolor='black', linewidth=0.8)
```

```
ax.axvline(0, color='black', linewidth=1)
ax.axvline(0.3, color='gray', linestyle='--', alpha=0.7, label='Medium Effect (|r| = 0.3)')
ax.axvline(-0.3, color='gray', linestyle='--', alpha=0.7)
ax.axvline(0.5, color='darkred', linestyle=':', alpha=0.7, label='Large Effect (|r| = 0.5)')
ax.set_title("Figure 13: Non-Parametric Mann-Whitney U Rank-Biserial Effect Sizes on Revenue",
fontsize=11, fontweight='bold', pad=12)
ax.set_xlabel("Rank-Biserial Correlation ($r_{rb}$)", fontsize=10, fontweight='bold')
ax.legend(frameon=True, fontsize=8.5, loc='lower right')
ax.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig('../figures/fig13_mann_whitney_effect_sizes.png', dpi=150)
plt.show()

# Chi-Square Test of Independence & Cramer's V for Categoricals
cat_test_cols = ['Month', 'OperatingSystems', 'Browser', 'Region', 'TrafficType', 'VisitorType',
'Weekend', 'High_Intent_Indicator']
chi2_results = []
N = len(df)

for col in cat_test_cols:
    contingency = pd.crosstab(df[col], df['Revenue'])
    chi2, p, dof, _ = stats.chi2_contingency(contingency)
    r, k = contingency.shape
    cramers_v = np.sqrt(chi2 / (N * min(r - 1, k - 1)))
    chi2_results.append({
        'Categorical Feature': col,
        'Chi2 Stat': chi2,
        'DoF': dof,
        'p-value': p,
        "Cramer's V": cramers_v,
        'Strength': 'Strong' if cramers_v >= 0.25 else ('Moderate' if cramers_v >= 0.15 else 'Weak')
    })

chi2_df = pd.DataFrame(chi2_results).sort_values(by="Cramer's V", ascending=False)
print("\n--- CHI-SQUARE & CRAMER'S V ASSOCIATION TABLE ---")
print(chi2_df.round(4).to_string(index=False))

# Figure 14: Cramer's V Association Bar Chart
fig, ax = plt.subplots(figsize=(8, 4.5))
bars = ax.barh(chi2_df['Categorical Feature'], chi2_df["Cramer's V"], color='#17B890',
edgecolor='black', linewidth=0.9)
for bar in bars:
    w = bar.get_width()
    ax.text(w + 0.01, bar.get_y() + bar.get_height()/2, f"{w:.3f}", va='center', fontsize=9,
fontweight='bold')

ax.set_title("Figure 14: Categorical Association Strength with Purchasing Outcome (Cramér's V)",
fontsize=11, fontweight='bold', pad=12)
ax.set_xlabel("Cramér's V Statistic", fontsize=10, fontweight='bold')
ax.set_xlim(0, 0.65)
ax.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig('../figures/fig14_cramers_v_heatmap.png', dpi=150)
plt.show()
```

Code Cell 11:

```
# Figure 11: Spearman Rank Correlation Heatmap
corr_features = num_features + ['Revenue']
corr_matrix = df[corr_features].corr(method='spearman')

fig, ax = plt.subplots(figsize=(11, 8.5))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm', center=0,
            vmin=-1, vmax=1, cbar_kws={'label': 'Spearman Correlation'},
            linewidths=0.5, linecolor='white', ax=ax, annot_kws={'size': 7.5})

ax.set_title('Figure 11: High-Resolution Spearman Rank Correlation Matrix of Session Features',
fontsize=12, fontweight='bold', pad=15)
ax.grid(False) # Strict rule: No grid lines over heatmaps
plt.xticks(rotation=45, ha='right', fontsize=8.5)
plt.yticks(fontsize=8.5)
plt.tight_layout()
```

```
plt.savefig('../figures/fig11_correlation_matrix_heatmap.png', dpi=150)
plt.show()

# Figure 12: Pairplot of Key Discriminative Features
sampled_df = df.sample(n=2500, random_state=SEED).copy()
sampled_df['Outcome'] = sampled_df['Revenue'].map({False: 'No Purchase', True: 'Purchase'})

pair_cols = ['PageValues', 'ExitRates', 'ProductRelated_Duration']
pair_plot = sns.pairplot(sampled_df, vars=pair_cols, hue='Outcome',
                         palette={'No Purchase': '#1E3D59', 'Purchase': '#FF6E40'},
                         plot_kws={'alpha': 0.5, 's': 20, 'edgecolor': 'none'},
                         diag_kind='kde', height=2.4)
pair_plot.fig.suptitle('Figure 12: Bivariate Pairplot of Key Predictive Features Stratified by
Outcome', fontsize=11, fontweight='bold', y=1.03)
plt.tight_layout()
plt.savefig('../figures/fig12_pairplot_key_features.png', dpi=150)
plt.show()

# Figure 15: Multivariate Interaction Surface (PageValues vs ExitRates)
df['PageValue_Tier'] = pd.cut(
    df['PageValues'],
    bins=[-0.001, 0.0, 10.0, 30.0, 400.0],
    labels=['0 (Zero Attribution)', '(0, 10] (Low)', '(10, 30] (Moderate)', '> 30 (High)']
)
df['ExitRate_Tier'] = pd.cut(
    df['ExitRates'],
    bins=[-0.001, 0.02, 0.05, 0.10, 0.201],
    labels=['[0, 0.02] (Very Low)', '(0.02, 0.05] (Low)', '(0.05, 0.10] (Moderate)', '(0.10, 0.20]
(High)']
)
interaction_table = df.pivot_table(index='ExitRate_Tier', columns='PageValue_Tier', values='Revenue',
aggfunc='mean') * 100

fig, ax = plt.subplots(figsize=(9, 4.8))
sns.heatmap(interaction_table, annot=True, fmt='.1f', cmap='YlGnBu', cbar_kws={'label': 'Conversion
Rate (%)'},
            linewidths=0.8, linecolor='white', ax=ax, annot_kws={'size': 10, 'weight': 'bold'})
ax.set_title('Figure 15: Multivariate Conversion Rate (%) across PageValues Tiers and ExitRates
Intervals', fontsize=11, fontweight='bold', pad=12)
ax.set_xlabel('PageValues Behavioral Attribution Tiers', fontsize=10, fontweight='bold')
ax.set_ylabel('ExitRates Level Intervals', fontsize=10, fontweight='bold')
ax.grid(False)
plt.tight_layout()
plt.savefig('../figures/fig15_multivariate_interaction_pagevalue_exit.png', dpi=150)
plt.show()
```

Code Cell 12:

```
# Interactive Session Explorer (Fry Stage 7)
plotly_sample = df.sample(n=1200, random_state=SEED).copy()
plotly_sample['Outcome'] = plotly_sample['Revenue'].map({False: 'No Purchase', True: 'Purchase'})

fig_plotly = px.scatter(
    plotly_sample,
    x='ExitRates',
    y='PageValues',
    size='Total_PageViews',
    color='Outcome',
    color_discrete_map={'No Purchase': '#1E3D59', 'Purchase': '#FF6E40'},
    hover_data=['Month', 'VisitorType', 'ProductRelated_Duration', 'Weekend'],
    title="Figure 17: Interactive Multi-Dimensional Session Explorer (Ben Fry Stage 7: Interact)",
    labels={'ExitRates': 'Session Exit Rate', 'PageValues': 'Google Analytics Page Value'},
    opacity=0.75
)

fig_plotly.update_layout(
    font=dict(family="DejaVu Sans", size=11),
    legend=dict(title="Conversion Status", orientation="h", yanchor="bottom", y=1.02, xanchor="right",
x=1)
)

fig_plotly.write_html('../figures/fig17_interactive_session_explorer.html')
print("Saved interactive HTML artifact to ../figures/fig17_interactive_session_explorer.html")
```

```
# Figure 16: Static High-Resolution Preview of Interactive Chart
fig, ax = plt.subplots(figsize=(8.5, 4.8))
sns.scatterplot(
    data=plotly_sample,
    x='ExitRates',
    y='PageValues',
    size='Total_PageViews',
    hue='Outcome',
    palette={'No Purchase': '#1E3D59', 'Purchase': '#FF6E40'},
    alpha=0.7,
    sizes=(20, 250),
    ax=ax,
    edgecolor='black',
    linewidth=0.5
)
ax.set_title('Figure 16: Static Preview of Multi-Dimensional Interactive Session Explorer',
fontsize=11, fontweight='bold', pad=12)
ax.set_xlabel('Session Exit Rate', fontsize=10, fontweight='bold')
ax.set_ylabel('Google Analytics Page Value', fontsize=10, fontweight='bold')
ax.legend(frameon=True, fontsize=8.5, loc='upper right')
plt.tight_layout()
plt.savefig('../figures/fig16_interactive_scatter_preview.png', dpi=150)
plt.show()
```

Code Cell 13:

```
# Preprocessing and Partitioning Pipeline
features_to_drop = ['Revenue', 'PageValue_Bin', 'ExitRate_Bin', 'PageValue_Tier', 'ExitRate_Tier']
X_full = df.drop(columns=[c for c in features_to_drop if c in df.columns])
y_full = df['Revenue'].astype(int).values

# 70/15/15 Stratified Partitioning
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X_full, y_full, test_size=0.15, stratify=y_full, random_state=SEED
)
val_adj_ratio = 0.15 / 0.85
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=val_adj_ratio, stratify=y_train_val, random_state=SEED
)

print(f"Dataset Partitions:")
print(f"  Training Split:   {X_train.shape[0]:,} sessions ({X_train.shape[0]/len(df)*100:.1f}%) |
Positives: {y_train.sum():,} ({y_train.mean()*100:.2f}%)")
print(f"  Validation Split: {X_val.shape[0]:,} sessions ({X_val.shape[0]/len(df)*100:.1f}%) |
Positives: {y_val.sum():,} ({y_val.mean()*100:.2f}%)")
print(f"  Held-out Test:    {X_test.shape[0]:,} sessions ({X_test.shape[0]/len(df)*100:.1f}%) |
Positives: {y_test.sum():,} ({y_test.mean()*100:.2f}%)")

cat_cols = ['Month', 'OperatingSystems', 'Browser', 'Region', 'TrafficType', 'VisitorType', 'Weekend',
'High_Intent_Indicator']
num_cols = [c for c in X_train.columns if c not in cat_cols]

for col in cat_cols:
    X_train[col] = X_train[col].astype(str)
    X_val[col] = X_val[col].astype(str)
    X_test[col] = X_test[col].astype(str)

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), cat_cols)
    ]
)

# Fit strictly on train ONLY
X_train_proc = preprocessor.fit_transform(X_train)
X_val_proc = preprocessor.transform(X_val)
X_test_proc = preprocessor.transform(X_test)

encoded_feature_names = num_cols +
list(preprocessor.named_transformers_['cat'].get_feature_names_out(cat_cols))
print(f"Transformed Feature Dimensionality: {X_train_proc.shape[1]} features post one-hot encoding.")
```

Code Cell 14:

```
# Deep Tabular MLP PyTorch Definition
class TabularMLP(nn.Module):
    def __init__(self, in_features, hidden1=64, hidden2=32, drop1=0.3, drop2=0.2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, hidden1),
            nn.BatchNorm1d(hidden1),
            nn.ReLU(),
            nn.Dropout(drop1),
            nn.Linear(hidden1, hidden2),
            nn.BatchNorm1d(hidden2),
            nn.ReLU(),
            nn.Dropout(drop2),
            nn.Linear(hidden2, 1)
        )
    def forward(self, x):
        return self.net(x).squeeze(1)

# Hyperparameter Tuning Grid (Selected Strictly on Validation Split)
tuning_configs = [
    {'h1': 32,  'h2': 16, 'drop1': 0.1, 'drop2': 0.1, 'lr': 1e-3, 'batch_size': 128},
    {'h1': 32,  'h2': 16, 'drop1': 0.2, 'drop2': 0.2, 'lr': 5e-4, 'batch_size': 128},
    {'h1': 64,  'h2': 32, 'drop1': 0.2, 'drop2': 0.1, 'lr': 1e-3, 'batch_size': 64},
    {'h1': 64,  'h2': 32, 'drop1': 0.3, 'drop2': 0.2, 'lr': 1e-3, 'batch_size': 128},
    {'h1': 64,  'h2': 32, 'drop1': 0.4, 'drop2': 0.3, 'lr': 5e-4, 'batch_size': 128},
    {'h1': 128, 'h2': 64, 'drop1': 0.3, 'drop2': 0.2, 'lr': 1e-3, 'batch_size': 128},
    {'h1': 128, 'h2': 64, 'drop1': 0.4, 'drop2': 0.3, 'lr': 5e-4, 'batch_size': 128},
    {'h1': 128, 'h2': 64, 'drop1': 0.5, 'drop2': 0.3, 'lr': 1e-4, 'batch_size': 64},
    {'h1': 96,  'h2': 48, 'drop1': 0.2, 'drop2': 0.2, 'lr': 1e-3, 'batch_size': 128},
    {'h1': 96,  'h2': 48, 'drop1': 0.3, 'drop2': 0.2, 'lr': 5e-4, 'batch_size': 128}
]

pos_weight = torch.tensor([(len(y_train) - sum(y_train)) / sum(y_train)], dtype=torch.float32)
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

tuning_results = []
print("Executing Hyperparameter Tuning on Validation Split (10 Configurations)...")

for idx, cfg in enumerate(tuning_configs):
    torch.manual_seed(SEED)
    m = TabularMLP(X_train_proc.shape[1], hidden1=cfg['h1'], hidden2=cfg['h2'], drop1=cfg['drop1'],
drop2=cfg['drop2'])
    opt = torch.optim.AdamW(m.parameters(), lr=cfg['lr'], weight_decay=1e-4)
    train_loader = DataLoader(
        TensorDataset(torch.tensor(X_train_proc, dtype=torch.float32), torch.tensor(y_train,
dtype=torch.float32)),
        batch_size=cfg['batch_size'], shuffle=True
    )

    # Train for 15 epochs for rapid validation search
    for ep in range(15):
        m.train()
        for bx, by in train_loader:
            opt.zero_grad()
            out = m(bx)
            loss = criterion(out, by)
            loss.backward()
            opt.step()

    m.eval()
    with torch.no_grad():
        v_logits = m(torch.tensor(X_val_proc, dtype=torch.float32))
        v_probs = torch.sigmoid(v_logits).numpy()

    val_pr_auc = average_precision_score(y_val, v_probs)
    val_roc_auc = roc_auc_score(y_val, v_probs)
    val_f1_05 = f1_score(y_val, (v_probs >= 0.5).astype(int))

    tuning_results.append({
        'Config_ID': f"C{idx+1}",
```

```
        'Architecture': f"[{cfg['h1']}-{cfg['h2']}]",
        'Dropout': f"({cfg['drop1']},{cfg['drop2']})",
        'LR': cfg['lr'],
        'Batch': cfg['batch_size'],
        'Val_F1': val_f1_05,
        'Val_PR_AUC': val_pr_auc,
        'Val_ROC_AUC': val_roc_auc
    })

tuning_df = pd.DataFrame(tuning_results).sort_values(by='Val_F1', ascending=False)
print("\n--- HYPERPARAMETER TUNING SEARCH RESULTS ---")
print(tuning_df.to_string(index=False))

best_cfg_row = tuning_df.iloc[0]
best_cfg_idx = int(best_cfg_row['Config_ID'].replace('C', '')) - 1
best_cfg = tuning_configs[best_cfg_idx]
print(f"\nOptimal Architecture Selected on Validation Split: {best_cfg_row['Config_ID']}
({best_cfg_row['Architecture']}, LR={best_cfg['lr']})")

# Figure 20: Hyperparameter Tuning Grid Chart
fig, ax = plt.subplots(figsize=(10, 4.5))
bars = ax.bar(tuning_df['Config_ID'], tuning_df['Val_F1'], color='#1E3D59', edgecolor='black',
width=0.55)
ax.axhline(best_cfg_row['Val_F1'], color='#FF6E40', linestyle='--', linewidth=1.5, label=f"Best Val F1
= {best_cfg_row['Val_F1']:.4f}")

for bar in bars:
    y = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, y + 0.005, f"{y:.3f}", ha='center', fontsize=8.5,
fontweight='bold')

ax.set_title('Figure 20: Hyperparameter Architecture Search (Evaluated Strictly on Validation Split)',
fontsize=11, fontweight='bold', pad=12)
ax.set_xlabel('Hyperparameter Configuration ID', fontsize=10, fontweight='bold')
ax.set_ylabel('Validation F1-Score (Threshold = 0.5)', fontsize=10, fontweight='bold')
ax.set_ylim(0, 0.75)
ax.legend(frameon=True, fontsize=9)
plt.tight_layout()
plt.savefig('../figures/fig20_hyperparameter_tuning_grid.png', dpi=150)
plt.show()
```

Code Cell 15:

```
# Final Tabular MLP Training with Optimal Hyperparameters
torch.manual_seed(SEED)
final_mlp = TabularMLP(
    X_train_proc.shape[1],
    hidden1=best_cfg['h1'], hidden2=best_cfg['h2'],
    drop1=best_cfg['drop1'], drop2=best_cfg['drop2']
)

optimizer = torch.optim.AdamW(final_mlp.parameters(), lr=best_cfg['lr'], weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=4)

train_loader = DataLoader(
    TensorDataset(torch.tensor(X_train_proc, dtype=torch.float32), torch.tensor(y_train,
dtype=torch.float32)),
    batch_size=best_cfg['batch_size'], shuffle=True
)

EPOCHS = 35
train_losses, val_losses = [], []
train_accs, val_accs = [], []

for epoch in range(EPOCHS):
    final_mlp.train()
    running_loss, correct, total = 0.0, 0, 0
    for bx, by in train_loader:
        optimizer.zero_grad()
        out = final_mlp(bx)
        loss = criterion(out, by)
        loss.backward()
        optimizer.step()
```

```
        running_loss += loss.item() * len(by)
        preds = (torch.sigmoid(out) >= 0.5).float()
        correct += (preds == by).sum().item()
        total += len(by)

    ep_train_loss = running_loss / total
    ep_train_acc = correct / total

    final_mlp.eval()
    with torch.no_grad():
        v_out = final_mlp(torch.tensor(X_val_proc, dtype=torch.float32))
        v_loss = criterion(v_out, torch.tensor(y_val, dtype=torch.float32)).item()
        v_preds = (torch.sigmoid(v_out) >= 0.5).float()
        v_acc = (v_preds == torch.tensor(y_val, dtype=torch.float32)).float().mean().item()

    scheduler.step(v_loss)
    train_losses.append(ep_train_loss)
    val_losses.append(v_loss)
    train_accs.append(ep_train_acc)
    val_accs.append(v_acc)

print(f"Training Complete! Final Epoch {EPOCHS}: Train Loss={train_losses[-1]:.4f}, Val
Loss={val_losses[-1]:.4f}, Val Acc={val_accs[-1]*100:.2f}%")

# Figure 18: Training vs Validation Loss Curves
fig, ax = plt.subplots(figsize=(8, 4.2))
ax.plot(range(1, EPOCHS + 1), train_losses, 'b-o', markersize=4, label='Training Loss (BCE Weighted)')
ax.plot(range(1, EPOCHS + 1), val_losses, 'r-s', markersize=4, label='Validation Loss')
ax.set_title('Figure 18: Deep Tabular MLP Training and Validation Loss Across Epochs', fontsize=11,
fontweight='bold', pad=12)
ax.set_xlabel('Training Epoch', fontsize=10, fontweight='bold')
ax.set_ylabel('Loss Value', fontsize=10, fontweight='bold')
ax.legend(frameon=True, fontsize=9)
plt.tight_layout()
plt.savefig('../figures/fig18_training_validation_loss.png', dpi=150)
plt.show()

# Figure 19: Training vs Validation Accuracy Curves
fig, ax = plt.subplots(figsize=(8, 4.2))
ax.plot(range(1, EPOCHS + 1), [a * 100 for a in train_accs], 'b-o', markersize=4, label='Training
Accuracy (%)')
ax.plot(range(1, EPOCHS + 1), [a * 100 for a in val_accs], 'g-^', markersize=4, label='Validation
Accuracy (%)')
ax.set_title('Figure 19: Deep Tabular MLP Classification Accuracy Across Epochs', fontsize=11,
fontweight='bold', pad=12)
ax.set_xlabel('Training Epoch', fontsize=10, fontweight='bold')
ax.set_ylabel('Accuracy (%)', fontsize=10, fontweight='bold')
ax.legend(frameon=True, fontsize=9)
plt.tight_layout()
plt.savefig('../figures/fig19_training_validation_accuracy.png', dpi=150)
plt.show()
```

Code Cell 16:

```
# Train Baseline Models
lr_model = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=SEED)
lr_model.fit(X_train_proc, y_train)

rf_model = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=SEED, n_jobs=-1)
rf_model.fit(X_train_proc, y_train)

# Validation Predictions for All Models
val_probs_lr = lr_model.predict_proba(X_val_proc)[:, 1]
val_probs_rf = rf_model.predict_proba(X_val_proc)[:, 1]

final_mlp.eval()
with torch.no_grad():
    val_probs_mlp = torch.sigmoid(final_mlp(torch.tensor(X_val_proc, dtype=torch.float32))).numpy()

# Calibrate Optimal Threshold on Validation Set Only
def find_best_threshold(y_true, y_probs):
    thresholds = np.linspace(0.05, 0.95, 91)
```

```
    f1_scores = [f1_score(y_true, (y_probs >= t).astype(int)) for t in thresholds]
    best_idx = np.argmax(f1_scores)
    return thresholds[best_idx], f1_scores[best_idx]

tau_lr, val_f1_lr = find_best_threshold(y_val, val_probs_lr)
tau_rf, val_f1_rf = find_best_threshold(y_val, val_probs_rf)
tau_mlp, val_f1_mlp = find_best_threshold(y_val, val_probs_mlp)

print(f"Validation Calibrated Decision Thresholds:")
print(f"  Logistic Regression: Optimal Threshold = {tau_lr:.2f} (Val F1 = {val_f1_lr:.4f})")
print(f"  Random Forest:       Optimal Threshold = {tau_rf:.2f} (Val F1 = {val_f1_rf:.4f})")
print(f"  Tabular Deep MLP:    Optimal Threshold = {tau_mlp:.2f} (Val F1 = {val_f1_mlp:.4f})")
```

Code Cell 17:

```
# Compute Test Set Predictions
test_probs_lr = lr_model.predict_proba(X_test_proc)[:, 1]
test_probs_rf = rf_model.predict_proba(X_test_proc)[:, 1]

with torch.no_grad():
    test_probs_mlp = torch.sigmoid(final_mlp(torch.tensor(X_test_proc, dtype=torch.float32))).numpy()

def evaluate_predictions(y_true, y_probs, threshold, model_name, thresh_type):
    y_pred = (y_probs >= threshold).astype(int)
    return {
        'Model': model_name,
        'Threshold_Type': thresh_type,
        'Threshold_Value': round(threshold, 2),
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred),
        'Recall': recall_score(y_true, y_pred),
        'F1_Score': f1_score(y_true, y_pred),
        'ROC_AUC': roc_auc_score(y_true, y_probs),
        'PR_AUC': average_precision_score(y_true, y_probs)
    }

metrics_list = [
    evaluate_predictions(y_test, test_probs_lr, 0.5, 'Logistic Regression', 'Default 0.5'),
    evaluate_predictions(y_test, test_probs_lr, tau_lr, 'Logistic Regression', f'Val-Tuned
({tau_lr:.2f})'),
    evaluate_predictions(y_test, test_probs_rf, 0.5, 'Random Forest', 'Default 0.5'),
    evaluate_predictions(y_test, test_probs_rf, tau_rf, 'Random Forest', f'Val-Tuned ({tau_rf:.2f})'),
    evaluate_predictions(y_test, test_probs_mlp, 0.5, 'Tabular Deep MLP', 'Default 0.5'),
    evaluate_predictions(y_test, test_probs_mlp, tau_mlp, 'Tabular Deep MLP', f'Val-Tuned
({tau_mlp:.2f})')
]

test_metrics_df = pd.DataFrame(metrics_list)
test_metrics_df.to_csv('../report/test_metrics.csv', index=False)
print("--- TEST SET BENCHMARKING METRICS TABLE ---")
print(test_metrics_df.round(4).to_string(index=False))

# Figure 21: Normalized Confusion Matrices Comparison
fig, axes = plt.subplots(1, 3, figsize=(14, 4.2))
models_for_cm = [
    ('Logistic Regression (Tuned)', test_probs_lr >= tau_lr, axes[0]),
    ('Random Forest (Tuned)', test_probs_rf >= tau_rf, axes[1]),
    ('Tabular Deep MLP (Tuned)', test_probs_mlp >= tau_mlp, axes[2])
]

for title, preds, ax in models_for_cm:
    cm = confusion_matrix(y_test, preds.astype(int), normalize='true') * 100
    sns.heatmap(cm, annot=True, fmt='.1f', cmap='Blues', ax=ax, cbar=False,
                xticklabels=['No Purchase', 'Purchase'], yticklabels=['No Purchase', 'Purchase'],
                annot_kws={'size': 11, 'weight': 'bold'})
    ax.set_title(title, fontsize=10, fontweight='bold')
    ax.set_xlabel('Predicted Label', fontsize=9, fontweight='bold')
    ax.set_ylabel('True Label', fontsize=9, fontweight='bold')
    ax.grid(False)

plt.suptitle('Figure 21: Normalized Test Set Confusion Matrices Comparison (%)', fontsize=12,
fontweight='bold', y=1.02)
plt.tight_layout()
```

```
plt.savefig('../figures/fig21_confusion_matrices_comparison.png', dpi=150)
plt.show()

# Figure 22: ROC Curves Comparison
fig, ax = plt.subplots(figsize=(8, 5))
for name, probs, col in [
    ('Logistic Regression', test_probs_lr, '#457B9D'),
    ('Random Forest', test_probs_rf, '#2A9D8F'),
    ('Tabular Deep MLP', test_probs_mlp, '#E63946')
]:
    fpr, tpr, _ = roc_curve(y_test, probs)
    auc_val = roc_auc_score(y_test, probs)
    ax.plot(fpr, tpr, label=f"{name} (AUC = {auc_val:.4f})", linewidth=2, color=col)

ax.plot([0, 1], [0, 1], 'k--', linewidth=1.2, label='Random Guessing (AUC = 0.50)')
ax.set_title('Figure 22: Receiver Operating Characteristic (ROC) Curves Comparison', fontsize=11,
fontweight='bold', pad=12)
ax.set_xlabel('False Positive Rate (1 - Specificity)', fontsize=10, fontweight='bold')
ax.set_ylabel('True Positive Rate (Recall / Sensitivity)', fontsize=10, fontweight='bold')
ax.legend(frameon=True, fontsize=9, loc='lower right')
ax.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig('../figures/fig22_roc_curves_comparison.png', dpi=150)
plt.show()

# Figure 23: Precision-Recall Curves Comparison
fig, ax = plt.subplots(figsize=(8, 5))
baseline_rate = y_test.mean()

for name, probs, col in [
    ('Logistic Regression', test_probs_lr, '#457B9D'),
    ('Random Forest', test_probs_rf, '#2A9D8F'),
    ('Tabular Deep MLP', test_probs_mlp, '#E63946')
]:
    prec, rec, _ = precision_recall_curve(y_test, probs)
    pr_auc_val = average_precision_score(y_test, probs)
    ax.plot(rec, prec, label=f"{name} (PR-AUC = {pr_auc_val:.4f})", linewidth=2, color=col)

ax.axhline(baseline_rate, color='k', linestyle='--', linewidth=1.2, label=f'Baseline Conversion Rate
({baseline_rate*100:.1f}%)')
ax.set_title('Figure 23: Precision-Recall Curves Under Severe Class Imbalance', fontsize=11,
fontweight='bold', pad=12)
ax.set_xlabel('Recall (Sensitivity)', fontsize=10, fontweight='bold')
ax.set_ylabel('Precision (Positive Predictive Value)', fontsize=10, fontweight='bold')
ax.legend(frameon=True, fontsize=9, loc='lower left')
ax.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig('../figures/fig23_pr_curves_comparison.png', dpi=150)
plt.show()

# Figure 24: Prediction Probability Score Distribution
fig, ax = plt.subplots(figsize=(8.5, 4.5))
sns.histplot(test_probs_mlp[y_test == 0], bins=35, stat='density', alpha=0.6, color='#1E3D59',
label='Actual Non-Purchasers (Class 0)', ax=ax)
sns.histplot(test_probs_mlp[y_test == 1], bins=35, stat='density', alpha=0.6, color='#FF6E40',
label='Actual Purchasers (Class 1)', ax=ax)
ax.axvline(tau_mlp, color='darkred', linestyle='--', linewidth=2, label=f'Calibrated Threshold (τ* =
{tau_mlp:.2f})')
ax.axvline(0.5, color='gray', linestyle=':', linewidth=1.5, label='Default Threshold (0.50)')

ax.set_title('Figure 24: Tabular Deep MLP Predicted Probability Density Distribution', fontsize=11,
fontweight='bold', pad=12)
ax.set_xlabel('Predicted Probability of Purchase P(Revenue=1)', fontsize=10, fontweight='bold')
ax.set_ylabel('Probability Density', fontsize=10, fontweight='bold')
ax.legend(frameon=True, fontsize=9)
plt.tight_layout()
plt.savefig('../figures/fig24_prediction_probability_distribution.png', dpi=150)
plt.show()
```

Code Cell 18:

```
# Permutation Feature Importance on Test Set
np.random.seed(SEED)
```

```
baseline_pr_auc = average_precision_score(y_test, test_probs_rf)
perm_importances = {}

# Evaluate key representative columns before one-hot expansion
key_eval_cols = [
    'PageValues', 'PageValue_Per_PageView', 'ExitRates', 'BounceRates',
    'ProductRelated', 'ProductRelated_Duration', 'Total_PageViews', 'Total_Duration',
    'Month', 'Administrative', 'Administrative_Duration', 'VisitorType', 'Weekend',
    'TrafficType', 'SpecialDay', 'High_Intent_Indicator'
]

for col in key_eval_cols:
    drops = []
    for _ in range(5):
        X_test_perm = X_test.copy()
        X_test_perm[col] = np.random.permutation(X_test_perm[col].values)
        X_test_perm_proc = preprocessor.transform(X_test_perm)
        perm_probs = rf_model.predict_proba(X_test_perm_proc)[:, 1]
        perm_pr_auc = average_precision_score(y_test, perm_probs)
        drops.append(baseline_pr_auc - perm_pr_auc)
    perm_importances[col] = np.mean(drops)

perm_df = pd.DataFrame(list(perm_importances.items()), columns=['Feature',
'Drop_in_PR_AUC']).sort_values(by='Drop_in_PR_AUC', ascending=True)

# Figure 25: Permutation Feature Importance Bar Chart
fig, ax = plt.subplots(figsize=(9, 5.5))
bars = ax.barh(perm_df['Feature'], perm_df['Drop_in_PR_AUC'], color='#1E3D59', edgecolor='black',
linewidth=0.8)

for bar in bars:
    w = bar.get_width()
    if w > 0.002:
        ax.text(w + 0.002, bar.get_y() + bar.get_height()/2, f"+{w:.3f}", va='center', fontsize=8.5,
fontweight='bold', color='#1E3D59')

ax.set_title('Figure 25: Test Set Permutation Feature Importance (Mean Drop in PR-AUC)', fontsize=11,
fontweight='bold', pad=12)
ax.set_xlabel('Degradation in Average Precision (PR-AUC)', fontsize=10, fontweight='bold')
ax.grid(axis='x', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig('../figures/fig25_permutation_feature_importance.png', dpi=150)
plt.show()
```

Code Cell 19:

```
# Export Comprehensive Summary Numbers JSON
summary_numbers = {
    'dataset': {
        'total_sessions': int(len(df_raw)),
        'total_features': int(df_raw.shape[1]),
        'engineered_features_count': 6,
        'missing_values': int(missing_count),
        'duplicate_rows': int(duplicate_count),
        'class_false_count': int(target_counts[False]),
        'class_true_count': int(target_counts[True]),
        'class_false_pct': float(round(target_pcts[False], 2)),
        'class_true_pct': float(round(target_pcts[True], 2)),
        'imbalance_ratio': float(round(imbalance_ratio, 2))
    },
    'partitions': {
        'train_count': int(X_train.shape[0]),
        'val_count': int(X_val.shape[0]),
        'test_count': int(X_test.shape[0]),
        'train_pos_pct': float(round(y_train.mean() * 100, 2)),
        'val_pos_pct': float(round(y_val.mean() * 100, 2)),
        'test_pos_pct': float(round(y_test.mean() * 100, 2)),
        'encoded_feature_dim': int(X_train_proc.shape[1])
    },
    'discrete_distribution_fits': {
        'product_related_mean': float(round(mean_views, 2)),
        'product_related_variance': float(round(var_views, 2)),
        'dispersion_index': float(round(dispersion_ratio, 2)),
```

```
        'poisson_aic': float(round(poisson_aic, 1)),
        'nbinom_aic': float(round(nb_aic, 1)),
        'delta_aic': float(round(poisson_aic - nb_aic, 1))
    },
    'continuous_distribution_fits': {
        name: {
            'aic': float(round(data['aic'], 1)),
            'ks_stat': float(round(data['ks'], 4)),
            'p_value': float(data['p'])
        }
        for name, data in cont_fits.items()
    },
    'hypothesis_testing': {
        'mann_whitney_top_feature': mw_df.iloc[0]['Feature'],
        'mann_whitney_top_r_rb': float(round(mw_df.iloc[0]['Rank-Biserial (r_rb)'], 4)),
        'mann_whitney_top_pval': float(mw_df.iloc[0]['p-value']),
        'cramers_v_top_cat': chi2_df.iloc[0]['Categorical Feature'],
        'cramers_v_top_val': float(round(chi2_df.iloc[0]["Cramer's V"], 4)),
        'cramers_v_month_val': float(round(chi2_df[chi2_df['Categorical Feature']=='Month']["Cramer's
V"].values[0], 4))
    },
    'hyperparameter_tuning': {
        'optimal_config_id': best_cfg_row['Config_ID'],
        'optimal_architecture': best_cfg_row['Architecture'],
        'optimal_val_f1': float(round(best_cfg_row['Val_F1'], 4))
    },
    'calibrated_thresholds': {
        'logistic_regression': float(round(tau_lr, 2)),
        'random_forest': float(round(tau_rf, 2)),
        'tabular_deep_mlp': float(round(tau_mlp, 2))
    },
    'test_metrics': {
        f"{row['Model']}_{row['Threshold_Type']}": {
            'accuracy': float(round(row['Accuracy'], 4)),
            'precision': float(round(row['Precision'], 4)),
            'recall': float(round(row['Recall'], 4)),
            'f1_score': float(round(row['F1_Score'], 4)),
            'roc_auc': float(round(row['ROC_AUC'], 4)),
            'pr_auc': float(round(row['PR_AUC'], 4))
        }
        for _, row in test_metrics_df.iterrows()
    }
}

json_path = '../report/summary_numbers.json'
with open(json_path, 'w') as f:
    json.dump(summary_numbers, f, indent=4)

print(f"Summary numbers successfully exported to {json_path}!")
```